NO-CODE MACHINE LEARNING FOR ENGINEERS, CHEMISTS AND ANALYSTS

ML Workbench

The complete User Guide — from your first CSV to models you can defend in a meeting.

Version 1.0 · Windows 64-bit desktop application

Written for readers with **zero** background in machine learning

All tutorials use the sample data that ships with the program

10 step-by-step tutorials 29 chapters Every tab, every field, every output file

FRONT MATTER

How to use this guide

This guide assumes you have never trained a model and that you do not intend to learn to program in order to try. Every idea is introduced in plain language, then immediately used on real data inside the program. There are no maths prerequisites beyond averages and percentages.

Three ways to read it

If you want to…	Read
Get a result today	Chapter 12 (“Your first run in eight clicks”), then Tutorial 1–3. Roughly 25 minutes at the keyboard.
Actually understand it	Part I straight through (chapters 1–9, about an hour), then every tutorial in Part III in order.
Look something up	Part IV is a reference: every tab field, every model, every output file, plus a 24-item troubleshooting list.

What “zero background” means here

Terms are defined the first time they appear and again in the glossary (chapter 29). When a number appears in a screenshot, the text explains where it came from. When a step can go wrong, the guide says what the wrong version looks like, because most beginner failures in this kind of tool are silent — the program happily gives you a number, it is just a meaningless number.

What you need

- A Windows 10 or 11 PC (64-bit). No Python, no admin rights, no internet connection.
- The `MLWorkbench.exe` file — one self-contained executable, already assembled (chapter 10).
- The four sample CSVs supplied with the program: `reactor_yield.csv`, `reactor_predict.csv`, `reactor_grade.csv`, `molecules.csv` — described column by column in appendix A.
- About 200 MB of free disk space for the program and the folders it creates.

The boxes you will see

> TIP
>
> Shortcuts and habits that save time, learned from doing this with real users.

> WHY IT MATTERS
>
> The reason behind a design decision, so you can judge for yourself instead of trusting the software.

> WATCH OUT
>
> A step where beginners commonly get a plausible-looking but wrong answer.

DO NOT DO THIS

A mistake that invalidates results rather than merely annoying you.

WHAT YOU SHOULD SEE

The exact numbers or text to compare against your own screen, so you know a step worked.

TRY IT

A two-minute exercise that teaches the point better than reading does.

A note on the numbers in this guide

Every screenshot and every number here was produced by running the shipped program on the shipped sample data. Your own results may differ in the third decimal place because some models start from random numbers, but the pattern — which model wins, how large the gain is, which feature dominates — reproduces reliably. When a number is fixed by design (row counts, column names, file names) the guide says so.

WHY IT MATTERS

If your numbers do not match at all, stop and check the settings before continuing: the seed, the fold count, the unticked columns and the target column all change the answer, and the guide states those settings each time.

Contents

PART I · FOUNDATIONS — MACHINE LEARNING WITHOUT THE JARGON

1 What this program does, in one page 7

2 A CSV is a table, and a table is a dataset 9

3 Features, target, and the shape of a learning problem 11

4 Regression and classification: the only two questions 12

5 Why a model that memorised the answers is worthless 13

6 Judging a model: the six metrics that matter 15

7 The baseline and the one-standard-error rule 17

8 Eight ways beginners fool themselves 18

9 The run folder: what reproducibility looks like 20

PART II · GETTING STARTED

10 Installing and launching 22

11 The window: a guided tour 24

12 Your first run in eight clicks 27

PART III · TUTORIALS, ONE PER FEATURE

13 Tutorial 1 — The Data tab and the health check 30

14 Tutorial 2 — Training your first model 34

15 Tutorial 3 — Reading the Results tab 37

16 Tutorial 4 — Predicting on new rows 41

17 Tutorial 5 — Optimising conditions 44

18 Tutorial 6 — Molecules: SMILES to features, then a model 48

19 Tutorial 7 — Classification, end to end 51

20 Tutorial 8 — Time series and grouped data 53

21 Tutorial 9 — The command line, command by command 55

22 Tutorial 10 — Fitting it into your working day 59

PART IV · REFERENCE

23 Field-by-field reference for every tab 62

24 Model catalogue 65

25 Run folder and file reference 68

26 Metric reference 70

27 Troubleshooting: 24 problems and their fixes 71

28 The data-preparation checklist 74

29 Glossary 75

APPENDICES

A The sample datasets, column by column 78

B Keyboard, menus and file locations 80

C How this program was built and verified 81

PART I

Foundations — machine learning without the jargon

Nine short chapters that give you the mental model the rest of the guide assumes. You do not need any of the mathematics. You do need to understand three sentences: what a model is, why a model that has seen the answers proves nothing, and why a dumb baseline is the most useful number on any results screen.

IN THIS PART

1 · What this program does, in one page

2 · A CSV is a table, and a table is a dataset

3 · Features, target, and the shape of a learning problem

4 · Regression and classification: the only two questions

5 · Why a model that memorised the answers is worthless

6 · Judging a model: the six metrics that matter

7 · The baseline and the one-standard-error rule

8 · Eight ways beginners fool themselves

9 · The run folder: what reproducibility looks like

CHAPTER 1

What this program does, in one page

ML Workbench reads a spreadsheet of things you have measured, learns the pattern that connects them, and then applies that pattern to rows you have not measured yet. That is the whole product.

A typical use: you have 140 experiments on a chemical reactor. For each one you recorded the temperature, the pressure, how much catalyst you charged and how long you waited — and, at the end, the yield you actually got. You now want to know what yield to expect at conditions you have not tried, and which of the four settings actually drives the yield.

You give the program the spreadsheet. It tries a dozen candidate patterns, checks each one against data it was not allowed to see during fitting, throws away the ones that only look clever, and hands you back:

- a single number saying how much of the variation in yield the pattern explains (and how much a naive guess would explain, for comparison);
- a ranked table of the candidate patterns, with the honest reason for the recommendation;
- a ranking of which columns matter most;
- a saved model file that turns four numbers into a predicted yield on demand;
- a written report you can attach to a review meeting.

What it deliberately will not do

It will not…	Because
clean your data silently	Every problem it finds is listed on the Data tab with a severity. You decide, and you can see what it decided to do about missing values.
hide the baseline	A model that cannot beat “predict the average” is not a model. The baseline is always the last row of every comparison table.
report only its best score	Every candidate model, its spread across folds and its holdout score stay on the table even after one is chosen.
tell you the result is good	“Good” depends on your process. The program tells you the size of the gain over the baseline and the uncertainty around it; you decide whether that is worth acting on.
phone home	There is no network code in the program at all. It runs offline, so it is safe for proprietary data on a locked-down machine.

The five things you will actually click

1 · Data

Open the CSV, choose what to predict, read the health check.

2 · Train

Pick models and validation settings, press Run training.

3 · Results

Read the comparison table and the recommendation.

4 · Predict

Apply the saved model to new rows.

5 · Optimise

Sweep the conditions and rank the outcomes.

6 · Molecules

Turn SMILES strings into numeric features.

WHY IT MATTERS

Notice what is not on that list: no “preprocessing” tab, no “encoder” step, no “apply the same transformation to new data” chore. In most beginner tools those steps live outside the model, so the day you use it on new data the transformations quietly differ and the predictions are nonsense. Here the imputation, encoding and scaling are saved inside the model file. A model is one object you can copy to another machine and it behaves identically.

CHAPTER 2

A CSV is a table, and a table is a dataset

Your data must be a table: rows are separate observations, columns are the things you measured, and each cell holds one value. In a file, that table is a **CSV** (comma-separated values), which is what Excel produces when you choose "Save as → CSV".

Rows, columns, cells

```
temperature_C,pressure_bar,catalyst_wt,residence_min,yield_pct,purity_pct
91.3,3.97,2.19,75.0,67.20,95.68
104.9,3.32,1.43,97.0,54.19,95.32
98.8,2.71,0.66,27.0,49.85,94.60
```

The first line is the **header** — the column names. Everything below it is data. This excerpt is the first three of 140 **rows** in `reactor_yield.csv`, one experiment per row. There are six **columns**. The program calls the whole table a **dataset** or **data frame**, and it never cares what the rows mean in your world — only that each row is a complete, independent observation.

The three column types you will meet

Type	Looks like	How the program uses it
Numeric	`91.3`, `0.66`, `75`	Used as-is. Empty cells are filled with the column median; models that need it (linear, support vector, k-nearest-neighbour, neural nets) also standardise the column so that a pressure in bar and a residence time in minutes carry comparable weight.
Categorical (text)	`R2`, `ethanol`, `day`	Converted to yes/no "dummy" columns — `reactor=R2` becomes 0 or 1, likewise `reactor=R1` and `reactor=R3`. Tree-based models skip this step; it is handled inside the model either way.
Identifier	`R-0001`, `batch-77`, a timestamp	Flagged, never used. A different value on every row teaches a model nothing and lets it "recognise" rows instead of learning physics.

What happens with a file that is not clean

Real files have empty cells, numbers stored as text, columns with trailing spaces, duplicated columns and rows with no answer. The program reads the file, reports every such problem on the Data tab with a colour-coded severity, and continues with a documented default: medians for numbers, most-frequent for text, rows without a target dropped. Chapter 13 walks through a deliberately broken file so you can see all nine findings it produces.

> TIP
>
> Save the CSV next to nothing else — avoid commas in the file name, avoid opening it in Excel and re-saving while the program has it open, and keep the header row as line one. If you must use Excel, "CSV UTF-8" is the safest export choice for files containing accented characters.

Encodings and separators, without the pain

Text files store characters with an **encoding**. Windows tools often produce `Windows-1252` or the Japanese `Shift-JIS`; modern tools produce `UTF-8`. Files from European Excel installations frequently separate columns with semicolons rather than commas. The program tries the common combinations automatically and tells you which one worked, in the grey line under the file name — for example `reactor_yield.csv (utf-8-sig)`. You never have to choose.

CHAPTER 3

Features, target, and the shape of a learning problem

Every supervised learning problem is the same sentence with two blanks: given these columns, what will that column be?

Consider `reactor_yield.csv`. You know temperature, pressure, catalyst charge and residence time before you run the experiment. You learn the yield only afterwards, when it is too late to change anything. So:

Name	In this example	Rule of thumb
Features also: inputs, predictors, X	`temperature_C`, `pressure_bar`, `catalyst_wt`, `residence_min`	Everything you know before the outcome exists, and are allowed to know.
Target also: label, response, y	`yield_pct`	The one column you want to predict. Exactly one per run.
Rows also: samples, observations, records	140 experiments	The examples the pattern is learned from and tested on. More is better; 30 is the practical minimum, and hundreds is comfortable.

The test that decides whether a column may be a feature

> WHY IT MATTERS
>
> Ask: **"At the moment I would use this model, would I already know this number?"** If the answer is no, the column cannot be a feature. In `reactor_yield.csv` there is a fifth column, `purity_pct`. Purity is measured on the same sample as yield, after the run. In a model that predicts yield before the run, purity is unavailable — and because purity and yield are correlated, including it produces a spectacular score that collapses the moment you try to use the model for real. Tutorial 1 shows this happening.

This is the single most common way a beginner model is wrong: not a maths error, but a column that would not exist at prediction time. The program cannot read your process, so it cannot catch this for you. It can catch the mechanical version of the same mistake — a column that is one-to-one with the target, or an ID — and it does, on the Data tab.

How many rows do you need?

The honest answer is "more than you have", but here is something workable. For the tree-based models in this program, tens of rows per feature is reasonable; hundreds is comfortable. With fewer than about 50 rows the cross-validation spread becomes so wide that differences between models are noise, and the program says so:

```
[warning] small-n: Only 12 rows. Cross-validation scores will have a wide spread.
          -> Treat differences under ~0.05 as ties and prefer the simpler model.
```

That warning is not decoration. When it appears, the correct response is to compare models with a shrug and pick the simplest one that beats the baseline.

CHAPTER 4

Regression and classification: the only two questions

If the target is a number on a continuous scale, the job is **regression**. If the target is a category, the job is **classification**. The maths underneath differs, the metrics differ, and mixing them up wastes an afternoon.

Regression

"What yield will I get?"
Target: `yield_pct` = 54.2
Output: a number, plus an honest error bar
Judged by: R^2, MAE, RMSE

Classification

"Will this batch be grade A?"
Target: `grade` = A / B / C
Output: a class, plus a probability per class
Judged by: accuracy, F1, balanced accuracy

Neither

"Group these spectra"
No target column at all — that is clustering, and it is out of scope for this program

The program decides for you, then lets you overrule it

When you pick a target, the program inspects the column and prints its inference under the target box: `Detected task: regression · range 32.87 … 72.46`. The rule is deliberately simple:

- text → classification;
- numbers with two or fewer distinct values → classification (a 0/1 flag);
- whole numbers with few distinct values → classification (grade labels stored as 1, 2, 3);
- anything else → regression.

The Train tab's **Task** dropdown is set to `auto` by default. Overrule it when you know better — for instance a "score" column holding only 1..5 that you genuinely want predicted as a quantity, not classified.

Two questions, the same reactor

Question	Target	What you do with the answer
Regression	`yield_pct` in `reactor_yield.csv`	Plan a campaign: "at 93 °C and 4.9 bar we expect about 69 %, not 60 %."
Classification	`grade` in `reactor_grade.csv`	Decide before the analysis finishes: "this batch is likely grade B, prepare the B packaging line." Classes here are A (49 rows), B (72), C (19).

WATCH OUT

Class labels are taken literally, including case and spaces. If one row says `Grade A` and another says `grade a`, the program sees three classes and no model will work. The health check flags both problems ("values that differ only by case", "leading/trailing spaces"); fix them in the file rather than hoping.

CHAPTER 5

Why a model that memorised the answers is worthless

Any model can score perfectly on the data it was trained on. It can do this by learning the pattern, or by memorising each row. From the score alone you cannot tell which — so the score on training data is never reported as a result.

The exam analogy, with the numbers that matter

A student who has seen the exam paper scores 100 %. That tells you nothing about the student. So we split the questions: fit the model on one part of the data and score it on a part it has never seen. This is a **holdout**.

```
140 rows
├── 112 rows  “training”  — the model may look at these while learning
└──  28 rows  “holdout”   — locked away, used once, at the very end
```

In the tutorial run (chapter 14) the holdout is 20 %, so 112 rows are used for tuning and 28 are held back. The locked rows are the only honest estimate of how the model will behave on the next batch you run.

When 28 rows is too few: cross-validation

A single holdout of 28 rows is a small sample; luck can move the score. So the 112 training rows are used more cleverly. They are cut into **folds** — five by default — and the model is trained five times: each time four folds are used to learn and the fifth to score. Every row is scored exactly once, in a fold where it was not used for learning.

```
fold 1:  [score] [train] [train] [train] [train]
fold 2:  [train] [score] [train] [train] [train]
fold 3:  [train] [train] [score] [train] [train]
fold 4:  [train] [train] [train] [score] [train]
fold 5:  [train] [train] [train] [train] [score]
```

You end up with five scores. Their **average** is the headline number and their **spread** is the uncertainty. Both appear in the results table — a model with a good average and a huge spread is not a safe choice.

Three ways to cut the folds, and when each is right

CV scheme	Use when	What it prevents
kfold (default)	Rows are independent: separate experiments, separate batches, separate samples.	Over-optimistic scores from scoring on the training rows.
group	Rows come in clusters — several measurements per batch, per patient, per wafer. Give the cluster column as the group column.	"Leakage between twins": the model recognises a batch it saw during learning and scores brilliantly on its sibling rows.
timeseries	Rows are in time order and the future must be predicted from the past.	Learning from tomorrow to predict yesterday — which looks great and is impossible in production.

Classification runs automatically use a **stratified** split: each fold keeps roughly the same mix of classes, so a rare grade is not accidentally left out of a fold.

> WHAT YOU SHOULD SEE
>
> In the log of a five-fold run, five lines like `r2 = 0.7192 (+/- 0.0644) in 12.4s`. The `+/-` is the cross-validation spread. The Results table shows the same pair as **R² (CV)** and **R² spread**.

CHAPTER 6

Judging a model: the six metrics that matter

A metric is a single number summarising how wrong the model is. You do not need to compute any of them by hand; you do need to know which ones to read and when a difference is real.

For regression

Metric	Plain meaning	How to read it
R^2	"How much of the variation in the target does the model explain?"	1.0 = perfect. 0.0 = no better than always predicting the average. **Negative** = worse than the average, which is what the baseline usually scores. 0.72 is a genuinely useful industrial model.
MAE	Mean absolute error: the average size of a miss, in the units of the target.	The number to quote to colleagues: "±2.9 percentage points of yield".
RMSE	Like MAE but punishes large misses harder.	If RMSE ≫ MAE, a handful of rows are badly wrong. Look at the residuals plot.
MAPE	Average error as a percentage of the true value.	Comparable across targets of different size. Avoid when values can be near zero.

For classification

Metric	Plain meaning	How to read it
Accuracy	Fraction of rows where the predicted class was right.	Misleading when classes are unbalanced. 72 of 140 rows are grade B, so always answering "B" already scores 51 %.
Balanced accuracy	Accuracy averaged over classes, so a rare class counts as much as a common one.	Use it when the rare class matters — the one you actually want to catch.
F1 macro	One number combining precision and recall, averaged over classes.	The headline metric in this program for classification. Ignore a model that raises accuracy while dropping F1.
Precision / recall	Of the rows you called A, how many were A? Of the rows that were A, how many did you catch?	They trade off. For a safety call, chase recall. For a costly action, chase precision.
ROC AUC	How well the model ranks rows by confidence, independent of the cut-off.	0.5 = coin flip, 0.9 = strong separation. Shown on the hold-out for multi-class runs.

When is a difference between two models real?

A useful rule of thumb, used by the program itself: if the difference between two averages is smaller than their spreads, the models are tied, and the simpler one wins. That is the one-standard-error rule of the next chapter, and it is why you will sometimes see the program recommend a model that is not at the top of the table.

TIP

Read the metrics in this order: **baseline first**, then the best model, then the recommended model, then the spread. If the baseline and the best model are close, stop — your data does not support a model, and no amount of clicking will change that.

CHAPTER 7

The baseline and the one-standard-error rule

Before you admire any model, ask what a lazy answer would score. That lazy answer is the baseline, and every comparison table in this program ends with it.

What the baseline actually is

- **Regression:** always predict the mean of the training target. Scores $R^2 = 0$ by definition on average, and slightly negative or positive on a fold depending on luck. In the tutorial run it scores **−0.144** across folds — negative, which is normal and is exactly what "no information" looks like.
- **Classification:** always predict the majority class. In `reactor_grade.csv` that is grade B, correct on 72 of 140 rows, giving a holdout accuracy of **0.50** and a macro F1 of **0.227**.

The gain over baseline is the result

The program states it explicitly after every run:

```
Baseline r2: -0.2531
→ RandomForest
   r2 = 0.7270 (+/- 0.0289) in 12.81s
Best score: RandomForest (0.7270)
Recommended: RandomForest (0.7270)
Gain over baseline: +0.9801
```

"Gain over baseline: +0.98" is the sentence to take to a meeting, not "R^2 0.73". The first is a statement about your process; the second is a number that means nothing without the context of what a trivial answer achieves on the same data.

The one-standard-error rule

Suppose gradient boosting averages 0.719 with a spread of 0.144, and random forest averages 0.719 with a spread of 0.064. Same average, very different reliability: gradient boosting's folds ranged wider, so its advantage on any given future dataset is a coin toss, while random forest is camping on a narrower shelf. Now suppose a third model averages 0.731 with a spread of 0.15 — it is "best" by a margin well inside the noise.

> WHY IT MATTERS
>
> The rule: **anything within one standard error of the best score is statistically indistinguishable from the best**. Among those, prefer the simplest model — fewer moving parts, easier to explain, less likely to break on next year's data. The program applies this automatically and prints the reason in the green line at the top of the Results tab: "RandomForest is both the best and the simplest within one standard error (0.719 ± 0.064 vs baseline -0.144). Use it."

The **complexity** column in the results table is the program's rough difficulty ranking of each model family, from 0 (plain linear regression) to 6 (gradient-boosted trees, and XGBoost when installed). It is used to break ties, nothing more.

CHAPTER 8

Eight ways beginners fool themselves

None of these produce an error message. All of them produce a nice table with a number on it. Read this chapter twice.

1 · Leakage through a co-measured column

Including `purity_pct` when predicting `yield_pct` looks like a triumph — the score jumps — and is useless, because purity is only known after the run. **Fix:** for each feature ask "would I know this before the outcome?" If not, untick it.

2 · An ID column used as a feature

`run_id` is unique per row. A flexible model can use it as a fingerprint and memorise. The health check reports `id-columns`; untick it.

3 · Tuning against the holdout

If you keep changing settings until the holdout score improves, the holdout has become a second training set and its score is now optimistic. **Fix:** choose the model using cross-validation only, then look at the holdout once, and report that.

4 · Reporting the best of twenty attempts

Click through twenty configurations and the winner looks better than it is, because you have implicitly selected on noise. **Fix:** decide in advance which models matter (the "recommended" preset exists for this), run once, and keep the whole table — the program saves it for you.

5 · Ignoring the spread

A model with 0.72 ± 0.30 is not better than one with 0.68 ± 0.02. The second will behave as advertised on new data; the first might do anything.

6 · Predicting where you have no data

Ask the model for yield at 300 °C when the experiments ran from 60 to 110 °C and it will answer confidently. That answer is extrapolation. In the Predict tab such rows are flagged in a `_warning` column and summarised in the warnings panel; in the Optimise tab the edge of the grid is marked in amber.

7 · Unbalanced classes and a useless "accuracy"

With 19 grade-C rows out of 140, a model can simply never predict C and still score 86 % accuracy — while being useless for the thing you cared about. **Fix:** read F1 macro and balanced accuracy, and tick **Balance classes** in the Train tab.

8 · Believing a model with no baseline

If you cannot say what the trivial answer scores, you cannot say whether the model is worth anything. This program makes that impossible to forget — the baseline is a row, not a footnote.

> TRY IT
>
> On the Data tab, tick `purity_pct` as a feature of `yield_pct` and run a training job. Then untick it and run again. Compare the two reports. The size of the jump you just saw is the size of the mistake a leaked column creates.

CHAPTER 9

The run folder: what reproducibility looks like

Every training run writes a timestamped folder. Six months later, that folder is the only reason anyone — including you — can tell what was actually done.

```
runs/
└── 20261110_192909_reactor_yield/
    ├── run.json                  every setting, metric, seed, library version, data hash
    ├── report.md                 the same story as readable text, table by table
    ├── metrics.csv               the results table, machine-readable
    ├── feature_importance.csv    permutation importance per feature
    ├── holdout_predictions.csv   actual vs predicted on the 28 untouched rows
    ├── models/RandomForest.pkl   imputation + encoding + scaling + model, in one file
    └── plots/
        ├── predicted_vs_actual.png
        ├── residuals.png
        ├── importance.png
        └── correlation.png
```

The **data hash** is the important, unglamorous part. It is a short fingerprint of the exact CSV file used — `7bf93e43dda9a53c` for the tutorial run. If someone re-runs the same settings on a file that has since been edited, the hash will differ, and the comparison is invalid before you even look at the score.

> WHAT YOU SHOULD SEE
>
> Open `report.md` and you will find, in order: what was asked, the data, the validation scheme, the comparison table with the baseline, the recommendation with its justification, the feature importances, the hyperparameters chosen, the environment (Python 3.11.9, scikit-learn 1.5.2, numpy 1.26.4), and a list of every file produced.

Nothing about this folder is proprietary or opaque: `.json` and `.md` open in any text editor, `.csv` opens in Excel, `.png` opens anywhere. The `.pkl` is the model and is only read by this program.

> TIP
>
> Zip the run folder and attach it to the review, not the screenshots. A reviewer who doubts your number can read `run.json` and see the seed, the fold count, the features and the baseline. This is what "reproducible" buys you in practice.

PART II

Getting started

Three chapters: putting the program on your machine, learning what every part of the window is for, and getting a first result without reading another word of theory. If you only have twenty minutes today, chapters 11 and 12 are the twenty minutes.

IN THIS PART

10 · Installing and launching

11 · The window: a guided tour

12 · Your first run in eight clicks

CHAPTER 10

Installing and launching

There is nothing to install. `MLWorkbench.exe` is a single, complete, self-contained program: Python, scikit-learn, matplotlib and RDKit are all inside it. Copy it somewhere sensible, double-click it, and you are running.

> WHAT YOU SHOULD HAVE
>
> One file — `MLWorkbench.exe` — about **113 MB** (118,498,091 bytes), version 1.0. There is no installer, no setup wizard, no dependency to fetch, and no second file it needs in order to start.

10.1 Where to put it

Location	Notes
`C:\MLWorkbench\`	A dedicated folder at the root of the drive. Clean, short path, easy to point IT at, easy to back up.
`Desktop`	Fine for a trial; you will end up with run folders next to your other user folders, which is also fine.
A USB stick	Works. The program writes nothing next to itself, so it leaves the stick untouched.
Network share	Works, but startup is slower because the exe is read over the network. Copy it locally instead.
`C:\Program Files\`	Possible but not recommended: Windows protects that folder, so a later update means an administrator. Keep the program out of it.

> WATCH OUT
>
> Copy the file, do not create a shortcut to a temporary location such as your **Downloads** folder — Windows may clear it out. If you want a desktop icon, right-drag the exe onto the Desktop and choose Create shortcut; the exe itself should live in a folder you control.

10.2 Starting it

1. **Double-click `MLWorkbench.exe`.** The first launch takes 10–20 seconds while Windows unpacks and caches the single-file build; later launches are quicker.
2. **Dismiss the SmartScreen prompt if it appears — More info → Run anyway** (see the table below for why it appears).
3. **The window opens on the Data tab** with an empty preview. The log at the bottom of the Train tab reads `Output folder for runs: C:\Users\<you>\MLWorkbench_runs`.
4. **Load a CSV** — press **Open CSV…** and choose `reactor_yield.csv`, or drag any CSV file onto the exe icon in Explorer, or use Open with → ML Workbench.

10.3 First-launch warnings you can safely pass

What you see	What it means and what to do
“Windows protected your PC” SmartScreen	The file is not code-signed (a certificate costs money per year). Click **More info → Run anyway**. This appears once per machine.
Antivirus quarantine	Some corporate scanners flag any unsigned `.exe`. Add an exclusion for the folder, or ask IT to whitelist the file by its SHA-256 fingerprint (section 10.4).
“Do you want to allow this app to make changes?”	You do not need administrator rights. If you see this prompt, cancel and run the exe from a normal user folder instead.
Slow first start (10–20 s)	The single-file build unpacks itself once per launch. Later launches are faster because Windows caches the file. If that is unacceptable, build the folder version described in appendix C — it starts in about 3 seconds.

10.4 Optional: confirm the file arrived intact

A 113 MB copy can be truncated by a browser, a proxy or a bad USB stick, and a truncated executable fails in confusing ways. To be certain — or because IT asked for a fingerprint — compare the file’s SHA-256 with the value below. In a console opened in the folder that holds the exe:

```
certutil -hashfile MLWorkbench.exe SHA256
expected: db745979458c7ee6c5b75bc263cbb9661c2b450814f3e18dbe76a3ac3f30aa7e
```

Identical hash means the file is complete and unmodified; a different hash means a corrupted or altered copy — obtain a fresh one. This step is optional: if the program starts and **Help → About** shows a version and a scikit-learn version, it is intact enough to use.

Where the program keeps its work

Runs are written to `MLWorkbench_runs` in your user folder (`C:\Users\<you>\MLWorkbench_runs`) unless you change it. The blue **Open runs folder** button in the bottom-right corner of the window opens it in Explorer from anywhere in the program; the File menu does the same.

> TIP
>
> Uninstalling is deleting the `.exe`. There is no registry entry, no service, no background process. Copy the exe onto a USB stick and it works on another PC with the same Windows version — models you already trained travel with their `.pkl` files and do not need the original data.

Running it without Windows or without the exe

If you have Python 3.11 on any operating system, the same program runs from source:

```
pip install -r requirements.txt
python run_mlworkbench.py          # the desktop window
python run_mlworkbench.py --help   # the command line, chapter 21
```

This is also the path if you want to read or change the source: the engine, the plots and the GUI are plain Python in `mlw/`.

CHAPTER 11

The window: a guided tour

One window, six tabs, one status bar. The tabs are deliberately numbered: the program wants you to work left to right, because each tab needs the previous one's output.

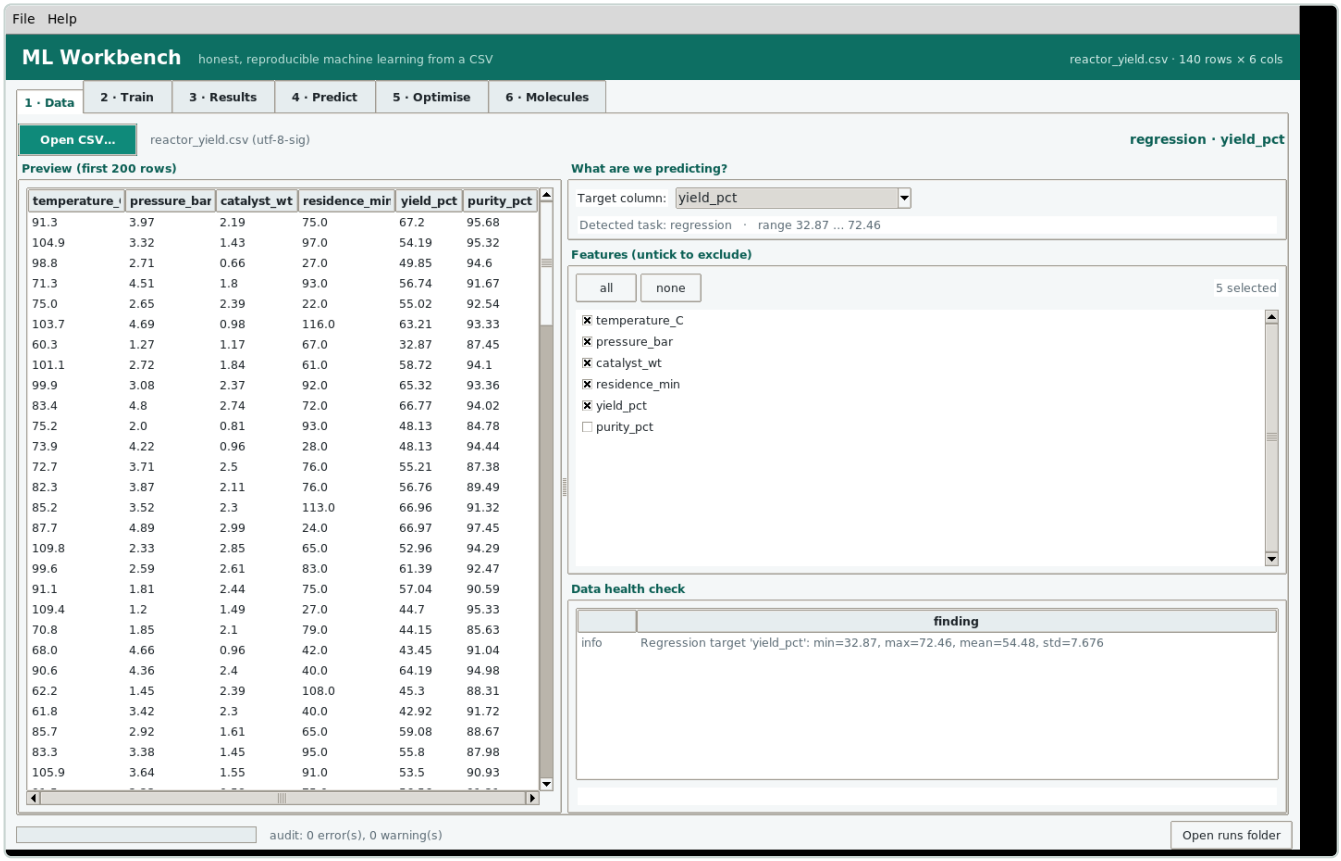


Figure 11.1 — The Data tab, the state you should reach before training anything. Left: the first 200 rows of the file exactly as read. Right: the target selector (here `yield_pct`, detected as regression), the feature list with tick boxes, and the data health check with one informational finding. The header on the right repeats the file name and shape: `reactor_yield.csv · 140 rows × 6 cols`.

Anatomy, from the top

Region	Purpose
Menu bar File · Help	**File**: open a CSV (Ctrl+O), open the runs folder, exit. **Help**: a one-screen quick guide, and About — which lists the exact library versions used.
Teal header	Product name and, on the right, the loaded file plus its shape. This is the fastest way to check you are looking at the file you think you are.
Tab strip	**1 · Data → 2 · Train → 3 · Results → 4 · Predict → 5 · Optimise → 6 · Molecules**.
Status bar bottom	A progress bar for long jobs, a one-line status (`done · recommended RandomForest`), and the **Open runs folder** button.

The six tabs, and what each one is for

Tab	You go here to…	You leave with…
1 · Data	load a table, choose the target, exclude columns, read the health check	a clean dataset and a target column
2 · Train	pick models, folds, holdout, seed, search budget; run	a run folder containing a model and a report
3 · Results	compare models honestly, read the recommendation and the importance ranking	a defensible choice of model
4 · Predict	apply the model to new rows and write a CSV	a prediction file, with warnings for rows you should not trust
5 · Optimise	sweep feature ranges to find the best (or worst) conditions	a ranked candidate table you can save
6 · Molecules	convert SMILES strings into numeric descriptors or fingerprints	a CSV you can train on, plus an applicability-domain check

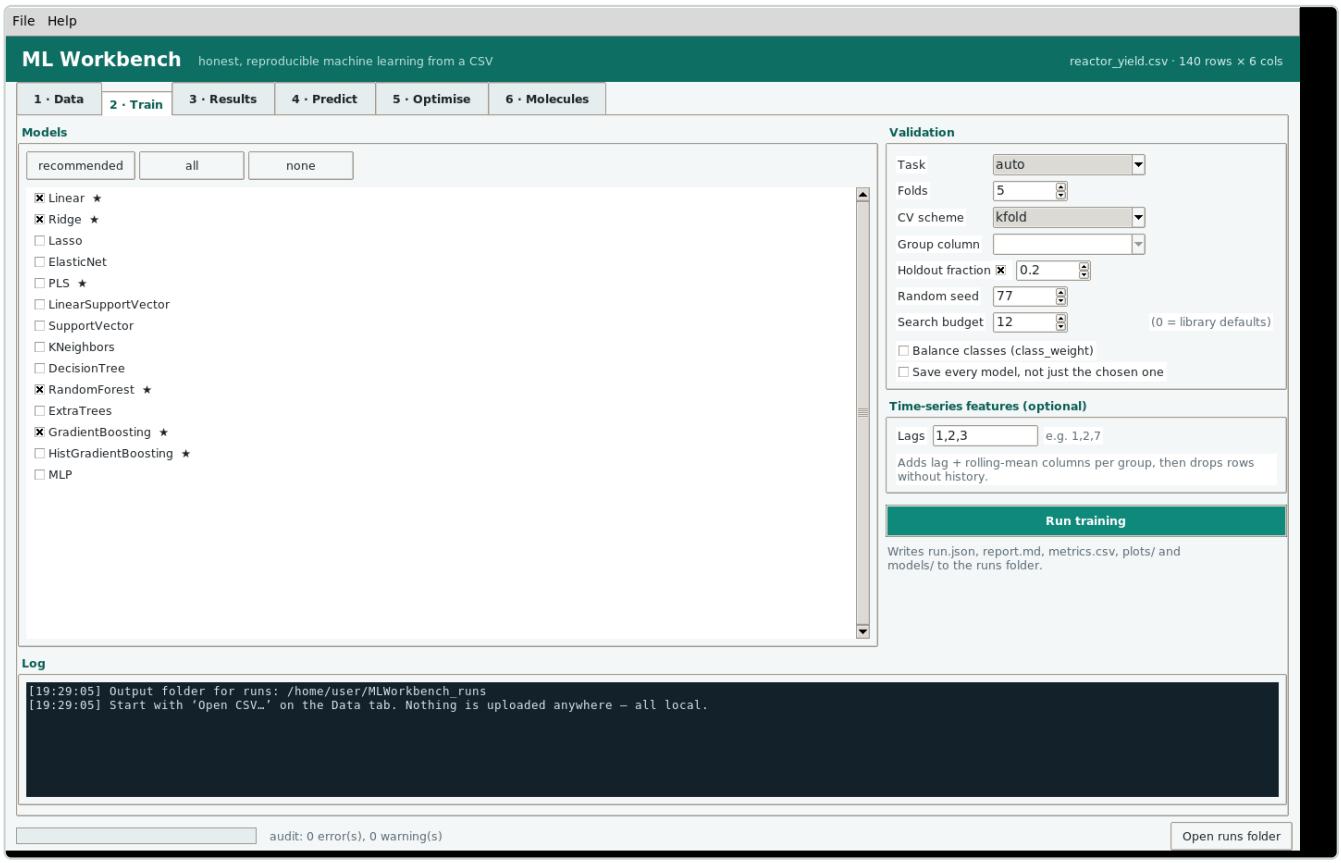


Figure 11.2 — The Train tab. Left: the model list with ★ marking the sensible default preset; the three buttons set the ticks for you (**recommended** / **all** / **none**). Right: validation settings — task, folds, CV scheme, group column, holdout, random seed, search budget — plus the class-balancing and save-all switches. Bottom: the log, where every step of a run is narrated with a timestamp.

The log is not decoration

The dark panel at the bottom of the Train tab is a transcript. It records the output folder, the data summary, the validation scheme, each model's score and time, the parameters chosen, the recom-

mendation, the importance ranking and where the run was written. When something takes longer than you expected, the log tells you which model is slow. When you want to reproduce a result by hand later, the log is the recipe.

> WHAT YOU SHOULD SEE
>
> On a fresh launch, the log reads: `Output folder for runs: C:\Users\<you>\MLWorkbench_runs` followed by `Start with 'Open CSV…' on the Data tab. Nothing is uploaded anywhere — all local.`

CHAPTER 12

Your first run in eight clicks

Two minutes, no theory. You will train a model that predicts reactor yield and read the result honestly. Every number below is what the shipped program produces on the shipped data.

1. **Click "Open CSV…"** (Data tab, top left). Choose `reactor_yield.csv` from the sample data folder.
2. **Check the header.** The right corner should read `reactor_yield.csv · 140 rows × 6 cols`, and the grey line under the button should end in `(utf-8-sig)` — that is the encoding it auto-detected.
3. **Set the target** to `yield_pct` in the Target column dropdown. Under it you should now see `Detected task: regression · range 32.87 … 72.46`, and a blue `info` row in the health check giving min, max, mean and standard deviation.
4. **Untick** `purity_pct` in the feature list on the right. This is the most important click in the tutorial: purity is measured after the run, so it cannot be an input. (Chapter 3 explains why; chapter 13 shows what happens when you forget.) You should be left with `4 selected`.
5. **Go to the Train tab** and press **recommended**. Six models get ticked: Linear, Ridge, PLS, RandomForest, GradientBoosting, HistGradientBoosting. For a first run, switch to a smaller set by ticking only **Linear**, **Ridge**, **RandomForest** and **GradientBoosting** — it finishes in about a minute instead of several.
6. **Leave the validation settings alone**: 5 folds, holdout ticked at 0.2, seed 77, search budget 12. They are good defaults; chapter 14 explains what each one does and when to change it.
7. **Click "Run training".** The progress bar fills, the log narrates. When the status bar says `done · recommended RandomForest`, the run has been written to disk.
8. **Go to the Results tab and read three things**, in this order: the green recommendation sentence at the top; the **BASELINE (mean)** row at the bottom of the table; the **What drives the prediction** ranking underneath.

WHAT YOU SHOULD SEE

With those four models, the results table is:

model	complexity	R² (CV)	R² spread	MAE	RMSE	test R²	test MAE
GradientBoosting	5	0.7192	0.1442	2.92	3.57	0.8072	2.51
RandomForest	4	0.7192	0.0644	3.00	3.79	0.7824	2.60
Ridge	1	0.4898	0.1412	4.06	5.07	0.6567	3.41
Linear	0	0.4859	0.1459	4.07	5.08	0.6611	3.40
BASELINE (mean)	−1	−0.1444	0.1838	6.05	7.78	−0.0488	5.52

and the green line reads "RandomForest is both the best and the simplest within one standard error (0.719 ± 0.064 vs baseline −0.144). Use it."

What just happened, in one paragraph

The program tried four kinds of pattern, scored each one five times on data it had not been allowed to learn from, and compared every score against a model that always answers “54.5 %”. The two tree-based models explain about 72 % of the variation in yield; the two linear models explain about 49 %. Because gradient boosting’s five scores bounced around four times as much as random forest’s, and averaged the same, random forest was recommended. On the 28 locked-away rows, random forest’s predictions were off by 2.6 percentage points on average. Temperature matters roughly four times as much as pressure, which matters three times as much as catalyst charge; residence time is nearly irrelevant in this dataset.

Where your work now lives

Open the runs folder (bottom-right button). You will find a folder named `<timestamp>_reactor_yield` containing the model, the report, the metrics table and four plots. Chapter 15 shows you how to read each of them; chapter 25 is the full file reference.

> TRY IT
>
> Run the same thing once more with the seed changed from 77 to 42. The scores will move slightly (third decimal place, occasionally the second). That movement is the honest noise in your data — memorise its size, because it tells you when a difference between two models is worth discussing.

PART III

Tutorials, one per feature

Ten tutorials, each covering one part of the program in the depth you need to use it unsupervised. Every tutorial runs on the sample data, states the settings it assumes, and shows the numbers you should get. Work through them in order the first time; afterwards they are reference material.

IN THIS PART

13 · The Data tab and the health check

14 · Training your first model

15 · Reading the Results tab

16 · Predicting on new rows

17 · Optimising conditions

18 · Molecules: SMILES to features, then a model

19 · Classification, end to end

20 · Time series and grouped data

21 · The command line, command by command

22 · Fitting it into your working day

TUTORIAL 1 · CHAPTER 13

The Data tab and the health check

Goal: understand every pixel of the Data tab, and learn to read a health check that finds nine different problems in one deliberately awful file.

13.1 What the tab shows

Element	What it does
Open CSV…	File picker. Accepts `.csv`, `.txt` and any file; encoding and separator are detected automatically. Ctrl+O does the same from any tab.
Grey line under the button	Full path plus the encoding that worked, e.g. `C:\MLWorkbench\reactor_yield.csv (utf-8-sig)`.
Preview table	The first 200 rows, read-only, with a horizontal scrollbar. Text is left-aligned, numbers right-aligned — a fast way to spot a column of numbers that is actually text.
Target column	The one column to predict. Choosing it automatically untick its own row in the feature list (a target cannot also be a feature) and triggers the health check.
Detected task line	Below the target box: `regression` or `classification`, plus the target's range. Chapter 4 explains the inference rule; you can overrule it on the Train tab.
Features (untick to exclude)	One checkbox per non-target column, with **all** / **none** buttons and a live count. This is where leakage is prevented, and it is the only place it can be.
Data health check	A list of findings. The left column is the severity (`error`, `warning`, `info`), the right column the finding itself. Click a row to see its hint in the grey line underneath.

13.2 The three severities

Severity	Meaning	What to do
error	Training will fail or be meaningless.	Fix the file. Do not proceed.
warning	Training will run, but the result may mislead.	Read the hint, decide consciously, and mention it when you present the result.
info	Context that is useful to know.	Nothing. It is there so you can quote the class balance or the target range.

13.3 A file with nine problems

Create the file below — or open `docs/guide_assets/messy_demo.csv` from the installation. It is a plausible export: an ID column, a duplicated temperature column, a solvent column with inconsistent spacing and case, a batch column, and a grade column with a missing value and one very rare class.

```
run_id,batch_id,temperature_C,pressure_bar,temperature_C_dup,catalyst_wt,reactor,solvent,grade
R-0001,B-71,91.3,3.97,91.3,2.19,R2, ethanol ,A
R-0002,B-71,104.9,3.32,104.9,1.43,R1,ethanol,A
R-0003,B-72,98.8,2.71,98.8,0.66,R2,Ethanol,B
...
R-0009,B-75,75.2,2.00,75.2,0.81,R1,water,
R-0007,B-74,60.3,1.27,60.3,1.17,R2,water,C
```

Load it with `Open CSV…` and set the target to `grade`. The health check now lists nine findings. Here they are, in the order the program produces them, with what each one means:

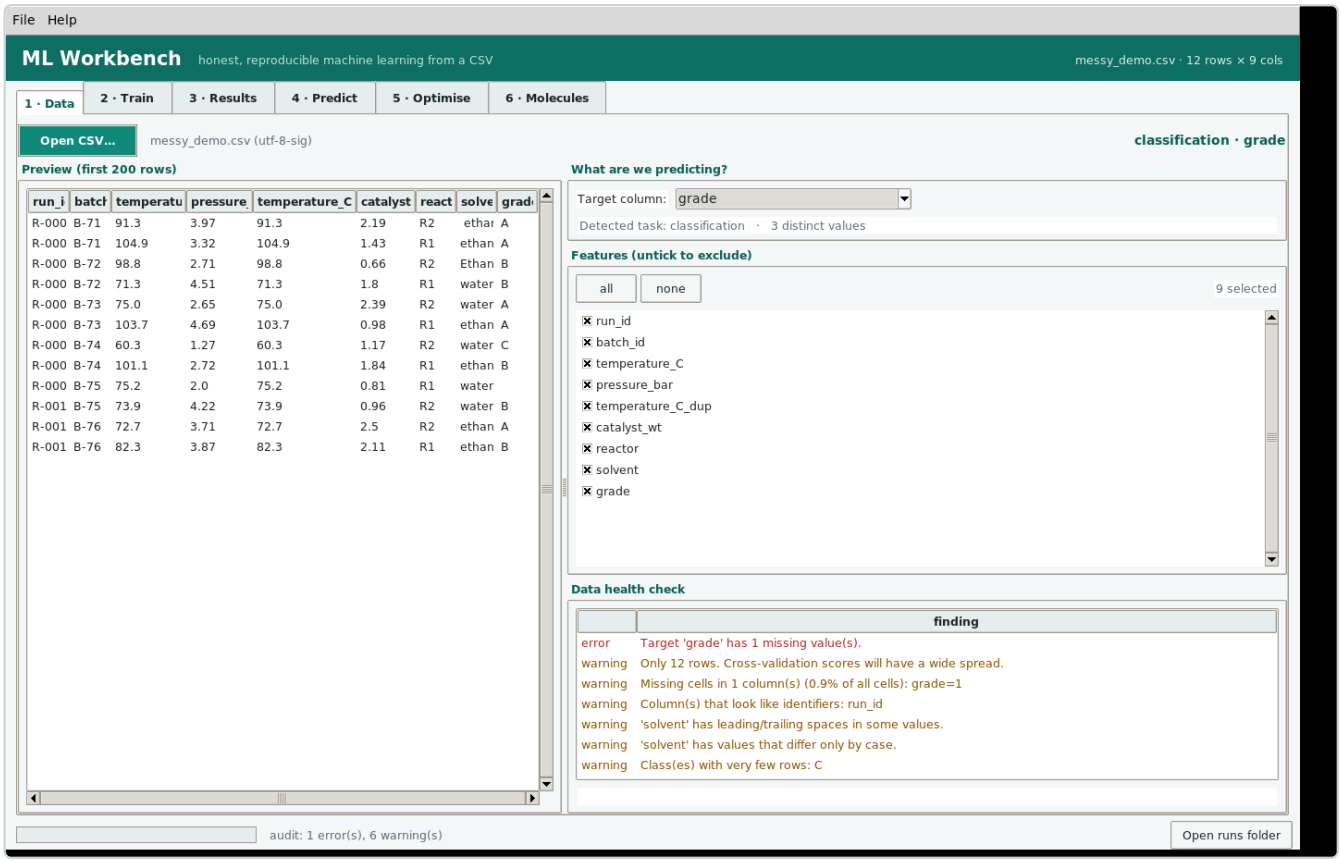


Figure 13.1 — The health check on `messy_demo.csv`. Nine findings: one error (missing target values), six warnings, two informational notes. The severity column is narrow and colour-coded; the finding text tells you precisely which column and how many rows.

Code	Level	Finding in this file	Meaning
`small-n`	warning	Only 12 rows	Cross-validation spread will be wide; treat differences under ~0.05 as ties.
`missing-values`	warning	`grade` =1	One empty cell. Numbers are imputed with the column median, text with the most frequent value — inside the model, so the same rule applies at prediction time.
`id-columns`	warning	`run_id`	Unique on every row. The model could memorise instead of learn. Untick it.
`whitespace-labels`	warning	`solvent`	`" ethanol "` and `"ethanol"` become two different categories. Strip the spaces.
`case-labels`	warning	`solvent`	`Ethanol` and `ethanol` become two categories. Normalise case.
`target-missing`	**error**	`grade` (1 row)	A row without a target cannot train anything. The program drops it and tells you; if the row was meant as a prediction case, it belongs in a separate file.
`class-balance`	info	A=5, B=5, C=1; majority 0.455	Quote the majority rate whenever you quote accuracy. Here a model that always says “A” is right 45 % of the time.
`rare-class`	warning	C has 1 row	Fewer than about five rows per class cannot be learned or validated. Consider merging or collecting more.
`redundant-features`	info	`temperature_C` / `temperature_C_dup` (1.00)	Perfectly correlated duplicate. Harmless for trees; it splits importance between twins and destabilises linear coefficients. Keep one.

WHY IT MATTERS

Notice what the program did not do: it did not silently rename `Ethanol`, delete `run_id`, or fill the missing grade. Automated cleaning that you cannot see is how analyses become unreproducible. It reported, and let you decide — and everything it does apply implicitly (imputation, encoding) is written into the model so the same rule is used later.

13.4 Which findings appear on a clean file

`reactor_yield.csv` produces exactly one finding — the informational target summary:

```
info   Regression target 'yield_pct': min=32.87, max=72.46, mean=54.48, std=7.676
```

That is the shape of a healthy start: no warnings, no errors, and you now know the target’s range, which you will need later to judge predictions and extrapolation.

TIP

Click each finding to read its hint before fixing anything. The hints name the exact remedy (“Rename them”, “Normalise case”, “Tick ‘Balance classes’…”), which is faster than guessing at what the tool wants.

TRY IT

Open `messy_demo.csv`, untick `run_id`, `batch_id` and `temperature_C_dup`, and run a classification job with only 12 rows. Watch how wide the cross-validation spread becomes. This is the practical meaning of “you need more data”.

TUTORIAL 2 · CHAPTER 14

Training your first model

Goal: understand every control on the Train tab, and know which ones you should change and which you should leave alone.

14.1 The model list

Left-hand panel: every model family available for the task you detected. The list changes when the target changes task — regression models for a numeric target, classifiers for a category target. A ★ marks the members of the **recommended** preset.

Button	Effect	When to use
recommended	Ticks six diverse, fast, well-behaved models.	Default for a first look at any dataset.
all	Ticks every model available for the task.	Second pass, when you want to know whether something exotic wins. Expect minutes, not seconds.
none	Clears all ticks.	Before hand-picking two models to compare head to head.

What each family is good at is in the model catalogue (chapter 24). The short version for a first pass: always include **Linear** or **Ridge** (they show you how much of the signal is merely linear), always include **RandomForest** (hard to break, handles mixed column types), and include **GradientBoosting** or **HistGradientBoosting** (usually the accuracy leader on tabular data, at the cost of needing more data).

14.2 The validation panel, field by field

Field	Default	What it does, and when to change it
Task	`auto`	Regression or classification, decided from the target (chapter 4). Override when the automatic guess is wrong, e.g. a numeric code you want predicted as a number.
Folds	5	How many cross-validation splits. More folds = more reliable estimate and more time; 5 is the standard compromise, 10 for small datasets, 3 when each model takes minutes.
CV scheme	`kfold`	`kfold` for independent rows. `group` when rows cluster (several rows per batch) — then choose the cluster column in **Group column**. `timeseries` when the row order is chronological and the future must be predicted from the past.
Group column	disabled	Enabled by the `group` scheme. The column whose values define a cluster; folds are cut so an entire cluster stays on one side.
Holdout fraction	0.2, ticked	Rows locked away before cross-validation starts, used once at the end for an unbiased number. 0.2 is a good default; 0.3 for small datasets; untick only when you cannot spare the rows — and then say so when you report.
Random seed	77	Fixes every random choice: fold assignment, forest bootstrapping, hyperparameter search order. Same seed + same data = same numbers, on any machine. Change it only to check that a conclusion survives a different split (see the Try-it in chapter 12).
Search budget	12	How many random hyperparameter combinations to try per model. **0** = use the library's defaults, fastest and least accurate. 12–30 is the useful range. Each trial costs a full inner cross-validation, so this is the main reason a run takes a minute instead of ten seconds.
Balance classes	off	Classification only: tells models that support it to weight rare classes more heavily. Tick whenever the minority class is the interesting one — defect, failure, grade C.
Save every model	off	Writes a `.pkl` for every candidate, not just the chosen one. Useful when you want to deploy "the linear one" for regulatory reasons while knowing the forest scored higher.

14.3 Time-series features

The **Lags** box adds lagged copies of the target and rolling means of those lags, per group, then drops rows that have no history. Entering `1,2,7` creates `lag_1`, `lag_2`, `lag_7` (and their rolling means) as features. Chapter 20 walks through it on a real time series.

14.4 Running and watching

1. Tick the models. The log reminds you which file is loaded and where runs will be written.
2. Press **Run training**. The button greys out; the status bar shows progress and the current stage ("cross-validating", "importance", "saving model"). The window stays responsive.
3. Read the log as it scrolls. Each model prints its tuned parameters and its score with spread. Slow models are visibly slow — that is information, not a bug.

4. When it finishes, the status bar reads `done · recommended <Model>` and the Results tab is automatically filled. The chosen model has also been pushed into the Predict and Optimise tabs, so you do not have to go looking for the `.pkl`.

WHAT YOU SHOULD SEE

For the four-model tutorial run, in this exact order in the log:

```
Data: 140 rows, 4 features (4 numeric, 0 categorical), target 'yield_pct' [regression]
Models: Linear, Ridge, RandomForest, GradientBoosting
CV: kfold, 5 folds, seed 77, holdout 20%
Holdout: 112 train / 28 test rows (test rows never used for tuning)
Baseline r2: -0.1444
→ Linear    r2 = 0.4859 (+/- 0.1459)
→ Ridge     r2 = 0.4898 (+/- 0.1412)
→ RandomForest r2 = 0.7192 (+/- 0.0644)
→ GradientBoosting r2 = 0.7192 (+/- 0.1442)
Best score: RandomForest (0.7192)
Recommended: RandomForest (0.7192)
Gain over baseline: +0.8636
```

14.5 How long should it take?

Dataset size	Models	Search budget	Typical time
140 rows × 4 features	4 (2 linear, 2 tree)	12	40–90 s
140 rows × 4 features	6 recommended	0	3–8 s
5,000 rows × 20 features	6 recommended	20	10–25 min
100,000 rows × 50 features	RandomForest + HistGradientBoosting	10	20–60 min (HistGradientBoosting is the fast one)

If a single model is much slower than its neighbours, it is usually **SupportVector** (scales badly with rows) or **MLP** (iterative training). Drop them for big datasets.

WATCH OUT

Do not close the window while training — the run folder is written at the end. If you must stop, wait for the status bar to return.

TUTORIAL 3 · CHAPTER 15

Reading the Results tab

Goal: read the results screen the way a reviewer would, in the right order, and know which numbers you are allowed to quote.

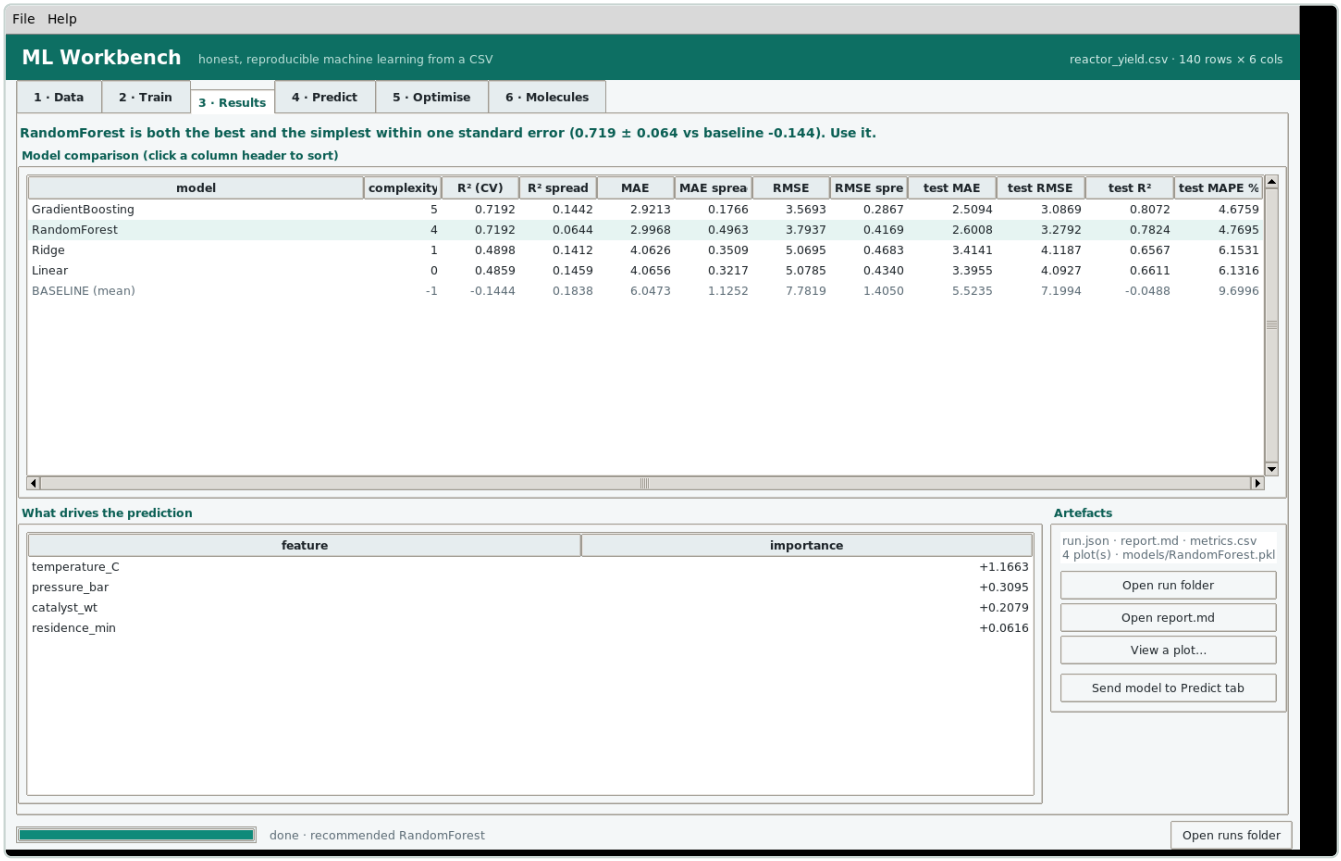


Figure 15.1 — The Results tab after the four-model regression run. Top: the recommendation, in a full sentence with its justification. Middle: the comparison table, baseline on the bottom row, with the holdout columns on the right. Bottom left: permutation importance. Bottom right: the artefacts panel — what exists on disk and four buttons to reach it.

15.1 Read it in this order

1. **The green recommendation sentence.** It names the model to use and gives the reason, including the numbers. If it says “gradient boosting scores highest, but random forest is within one standard error and simpler”, you are being told the difference is noise.
2. **The BASELINE row.** The last row, always. If the best model is not clearly better than this, there is no result to report.
3. **The R² (CV) column and the R² spread column.** Average, then uncertainty. A model with a good average and a spread of 0.3 is a coin flip.
4. **The test (holdout) columns.** The unbiased confirmation. It is normal for these to differ from the CV numbers by a few points; it is a red flag if they are dramatically worse (overfitting) or dramatically better (luck on 28 rows).
5. **What drives the prediction.** The importance ranking, which is where the domain conversation starts.

15.2 Every column of the comparison table

Column	Regression	Classification
model	The model family. `BASELINE` is the trivial reference: mean prediction, or majority class.	
complexity	Rough difficulty rank, 0 (linear) to 6 (boosted trees). Used to break ties under the one-standard-error rule.	
R² (CV) / F1 macro	Average R² over the folds — the primary metric.	Average macro F1 over the folds — the primary metric.
… spread	Standard deviation across folds: the honest uncertainty of that average.	
MAE / RMSE	Average miss, in target units; and the same with large errors punished harder.	Not shown; classification reports accuracy, balanced accuracy, F1 macro, F1 weighted.
test R², test MAE, test RMSE, test MAPE %	The holdout scores — computed once, on rows never used for tuning.	`test acc`, `test bal. acc`, `test F1`, `test F1 wtd`, precision/recall macro, ROC AUC.

15.3 Sorting and drilling in

- **Click any column header** to sort by it — sort by R² spread ascending to find the most reliable model, which is rarely the same as the highest average.
- **Click a row** (not a header) to open a detail dialog: the tuned hyperparameters, the complexity rank, the number of fitted parameters, the fit time, and every fold's individual score. This is the fastest way to answer "what exactly did it choose?".
- **Send model to Predict tab** copies the chosen model's path into the Predict and Optimise tabs, saving you a file dialog.

15.4 The four artefacts buttons

Button	Opens
Open run folder	The whole timestamped folder in Explorer.
Open report.md	The written report — settings, table, recommendation, importances, environment, file list. Attach this to your review.
View a plot…	Asks which plot by name, then opens the PNG. Names are `predicted_vs_actual`, `residuals`, `importance`, `correlation`, and `confusion_matrix` for classification.
Send model to Predict tab	Hands the `.pkl` over, as above.

15.5 The three plots that change your mind

Plot	What to look for
predicted_vs_actual	A cloud hugging the diagonal. Curvature means the model is systematically low or high in part of the range; a flat cloud at low values means it cannot resolve them.
residuals	Error against prediction. You want a shapeless band around zero. A funnel (errors growing with the prediction) says the model is unreliable at the top end; an arc says a transformation is missing — a job for the domain expert, not more clicking.
correlation	Feature-versus-feature correlation. Two blocks of near-1.0 tell you the features are redundant (the health check also reports this), which matters for interpreting linear coefficients.

15.6 Feature importance, honestly

The bottom-left panel shows **permutation importance**: how much the score degrades when one column's values are shuffled. In the tutorial run:

```
temperature_C   +1.166
pressure_bar    +0.310
catalyst_wt     +0.208
residence_min   +0.062
```

Read it as "how much this dataset relies on the column", not as physics. It is computed on the fitted model, so a column the model ignored scores near zero even if chemistry says it matters. A column scoring negative means shuffling it helped — a sign the model was overfitting on it.

> TIP
>
> Importance rankings are the best answer to "so what?" — and the easiest thing to over-claim. Report them with their uncertainty: the same experiment with a different seed will reorder the bottom two places. Only large differences are worth talking about, and only a designed experiment can prove causality.

15.7 A classification results screen

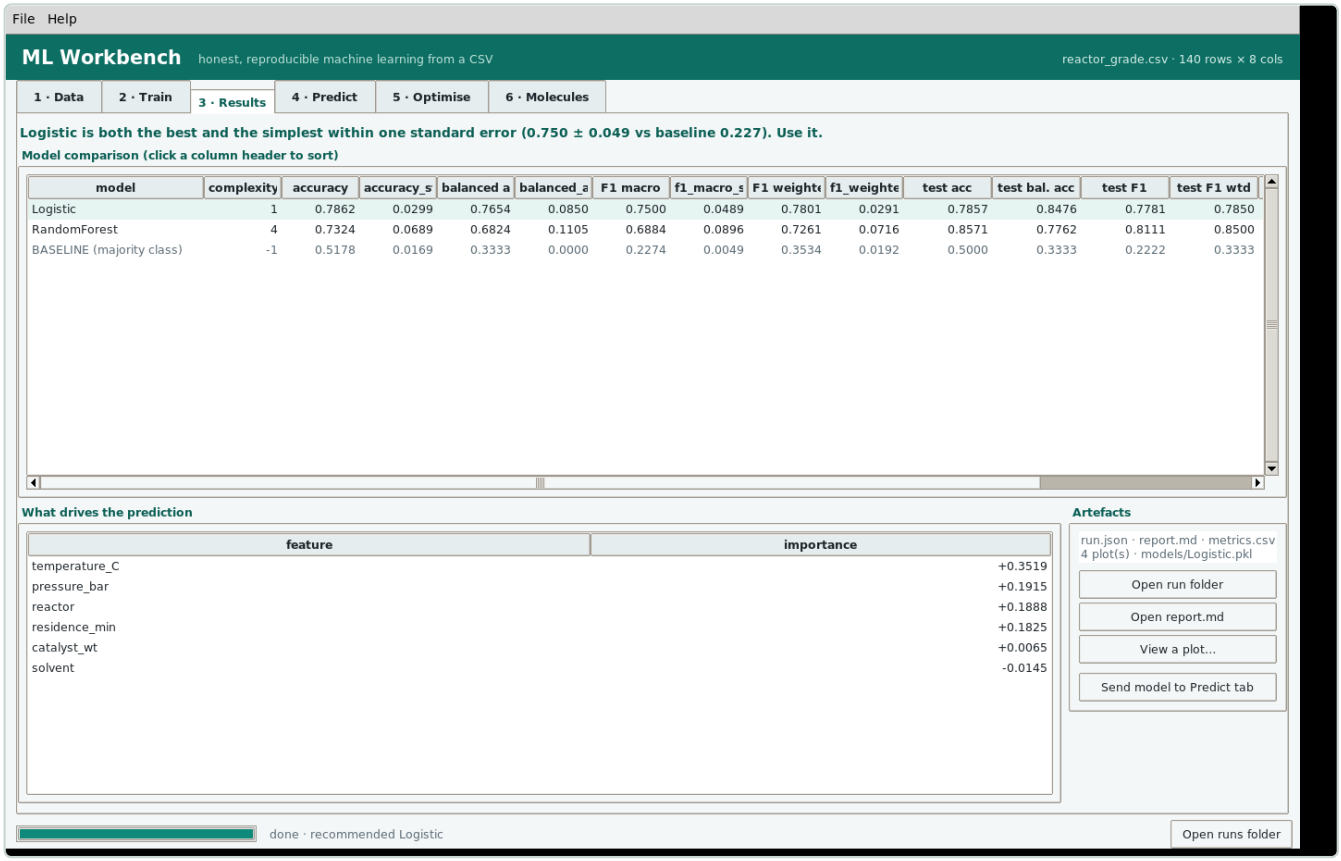


Figure 15.2 — The same tab for a classification target. `reactor_grade.csv`, target `grade` (A/B/C), 6 features, class balancing on. Logistic regression wins on macro F1 (0.750) while random forest wins on holdout accuracy (0.857) — a textbook trade-off, and the recommendation line tells you which one the program chose and why.

> WHAT YOU SHOULD SEE
>
> In that run: Logistic — accuracy 0.786, balanced accuracy 0.765, F1 macro 0.750, holdout accuracy 0.786, holdout ROC AUC 0.936. RandomForest — accuracy 0.732, F1 macro 0.688, holdout accuracy 0.857. Baseline (majority class “B”) — accuracy 0.518, F1 macro 0.227. Both models are a large, real improvement over the baseline; note how much higher the holdout accuracy is than the CV accuracy for the forest, which is a reminder that 28 rows is a small sample.

TUTORIAL 4 · CHAPTER 16

Predicting on new rows

Goal: apply a saved model to a table of new conditions, understand every warning the Predict tab can raise, and know which of its rows you are allowed to believe.

16.1 The three inputs

Field	What it needs
Model (.pkl)	Any model written by a training run. Already filled in if you pressed Send model to Predict tab. The model carries its own list of required columns, its imputation rules, its categories and its training ranges.
New data (CSV)	A table with the same feature column names. Extra columns are ignored; missing ones are an error and are named. The target column is not needed — that is the point.
Output CSV	Where to write the predictions. Leave blank and the program proposes `<yourfile>_predictions.csv` next to your input.

1. Choose the model. If you just trained, click **Send model to Predict tab** on the Results tab instead of browsing.
2. Choose `reactor_predict.csv` — 10 rows, the four feature columns, no target.
3. Press **Validate & predict**.
4. Read the report panel, then the preview table, then open the output CSV.

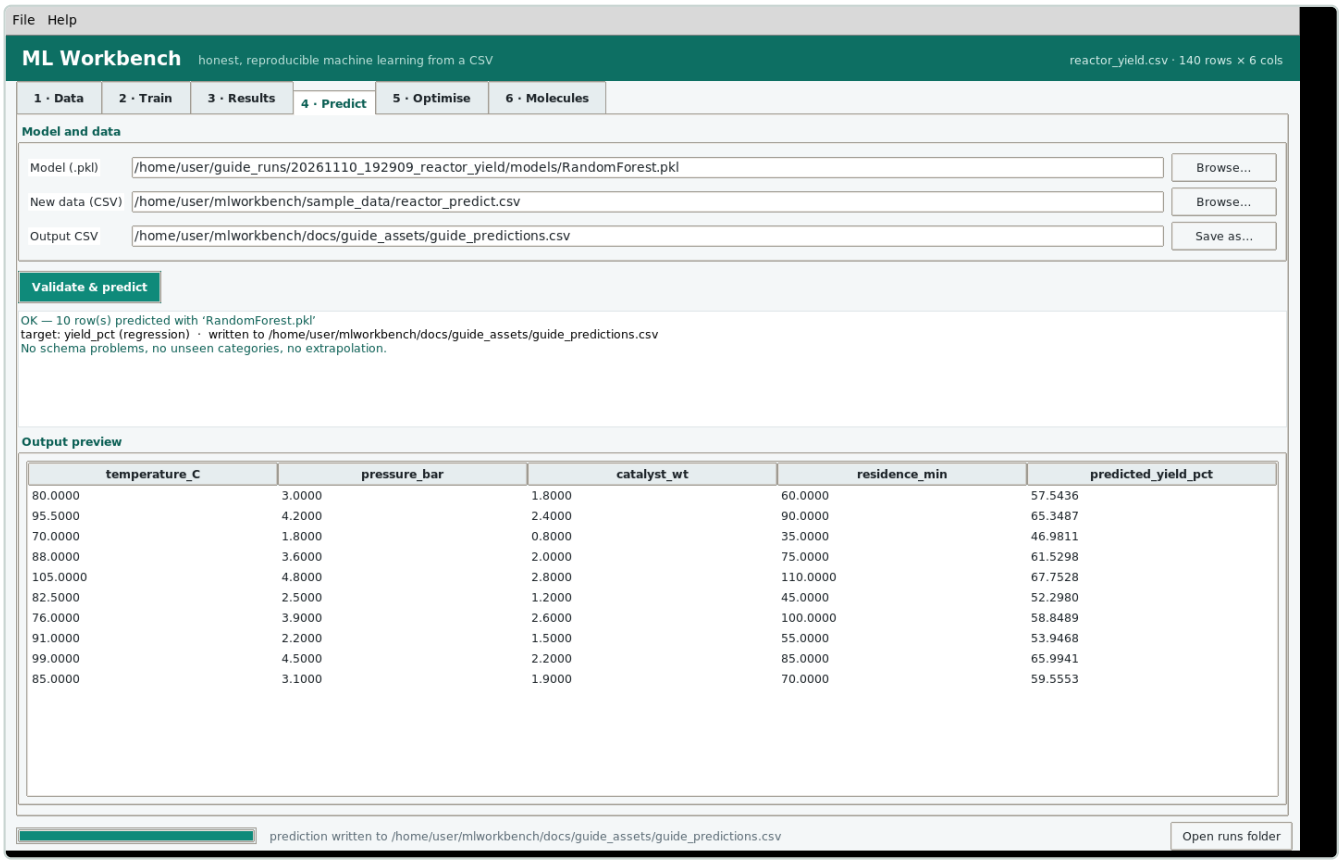


Figure 16.1 — The Predict tab. The report panel states how many rows were predicted, with which model, into which file, and — crucially — whether the schema matched, whether any category was unseen, and whether any row was extrapolation. The preview shows the first 100 rows of the output.

WHAT YOU SHOULD SEE

```
OK — 10 row(s) predicted with 'RandomForest.pkl'
target: yield_pct (regression)  ·  written to ...\reactor_predict_predictions.csv
No schema problems, no unseen categories, no extrapolation.
```

The output file has five columns — the four inputs plus `predicted_yield_pct` — and the first rows read 57.54, 65.35, 46.98, 61.53, 67.75 %.

16.2 The five things the report can tell you

Report line	Meaning and response
Missing columns	The new file lacks a feature the model needs. It is named in the report and the run stops (exit code 2 on the command line). Fix the file — do not proceed with a model that is silently fed medians for a real feature.
Unseen categories	A text column contains a value never seen in training (e.g. a new reactor `R4`). The model treats it as all-zeros in the dummy encoding, i.e. "none of the known reactors". Predictions still come out; treat the affected rows as guesses.
EXTRAPOLATION	One or more rows sit outside the range seen in training for a numeric feature. The report gives the column, the count and the training range. These rows are flagged in a `_warning` column of the output file. This is a warning, not an error: you may deliberately explore outside the range, but you must not quote those numbers as predictions.
Empty cells imputed	The new file has blanks. They are filled using the training-time medians and modes stored inside the model, which is the correct way — not with the new file's own values.
Text converted to number	A column arrived as text (Excel formatting) and was coerced. Harmless, but it usually means the file was exported oddly; check a couple of rows.

16.3 Classification output

For a classifier the output contains the predicted class plus one probability column per class:

```
temperat-
ure_C,pres-
sure_bar,catalyst_wt,residence_min,reactor,solvent,predicted_grade,proba_A,proba_B,proba_C
91.3,3.97,2.19,75.0,R2,ethanol,A,0.8213,0.1742,0.0045
```

The probabilities are what makes a classifier usable in practice: "A with 0.82" and "A with 0.34" carry different operational meaning even though both predict A. A column that does not exist in the output is the one your model cannot produce — a linear SVM, for instance, has no probabilities unless it was built with them.

> WATCH OUT
>
> Predictions are only as good as the model's coverage. A useful habit: after predicting, sort the output by the predicted value and look at the extremes. If the highest predictions are all in rows flagged as extrapolation, your best-case numbers are the least trustworthy ones — a trap that shows up in Optimise too (chapter 17).

> TRY IT
>
> Copy `reactor_predict.csv`, change one temperature to `300`, and predict again. The report turns red with an EXTRAPOLATION line naming `temperature_C` and the training range 60.2…109.8, and that row gains a `_warning` cell. That is the program refusing to be confident where it has no evidence.

TUTORIAL 5 · CHAPTER 17

Optimising conditions

Goal: use a trained model backwards. Instead of “given these conditions, what yield?”, ask “which conditions give the best yield?” — and learn why the top of that list needs reading carefully.

17.1 The idea, and its one great limitation

The Optimise tab builds a grid of conditions, asks the model to predict every point in it, and sorts the results. It is fast and it is genuinely useful for planning an experiment campaign. It has one limitation you must keep in mind: **the model only knows what it was trained on**. If you ask for a temperature outside the experimental range, the model will extrapolate smoothly and confidently, and the program will mark those rows — in amber at the edge, in red outside.

17.2 From model to grid in four steps

1. Make sure a model is in the **Model (.pkl)** box (the Results tab can push it there).
2. Press **Load ranges**. The program reads the model and pre-fills one row per numeric feature with its **training minimum, maximum and 6 steps**, plus one row per categorical feature pre-filled with its known levels separated by `|`.
3. Edit any min / max / steps. Steps is the number of intervals, so 6 means 7 values including both ends. Choose the objective: **maximise** or **minimise**, and how many rows to show.
4. Press **Run optimisation**, read the ranked table, then **Save results as CSV…** to keep the whole grid (not just the visible top).

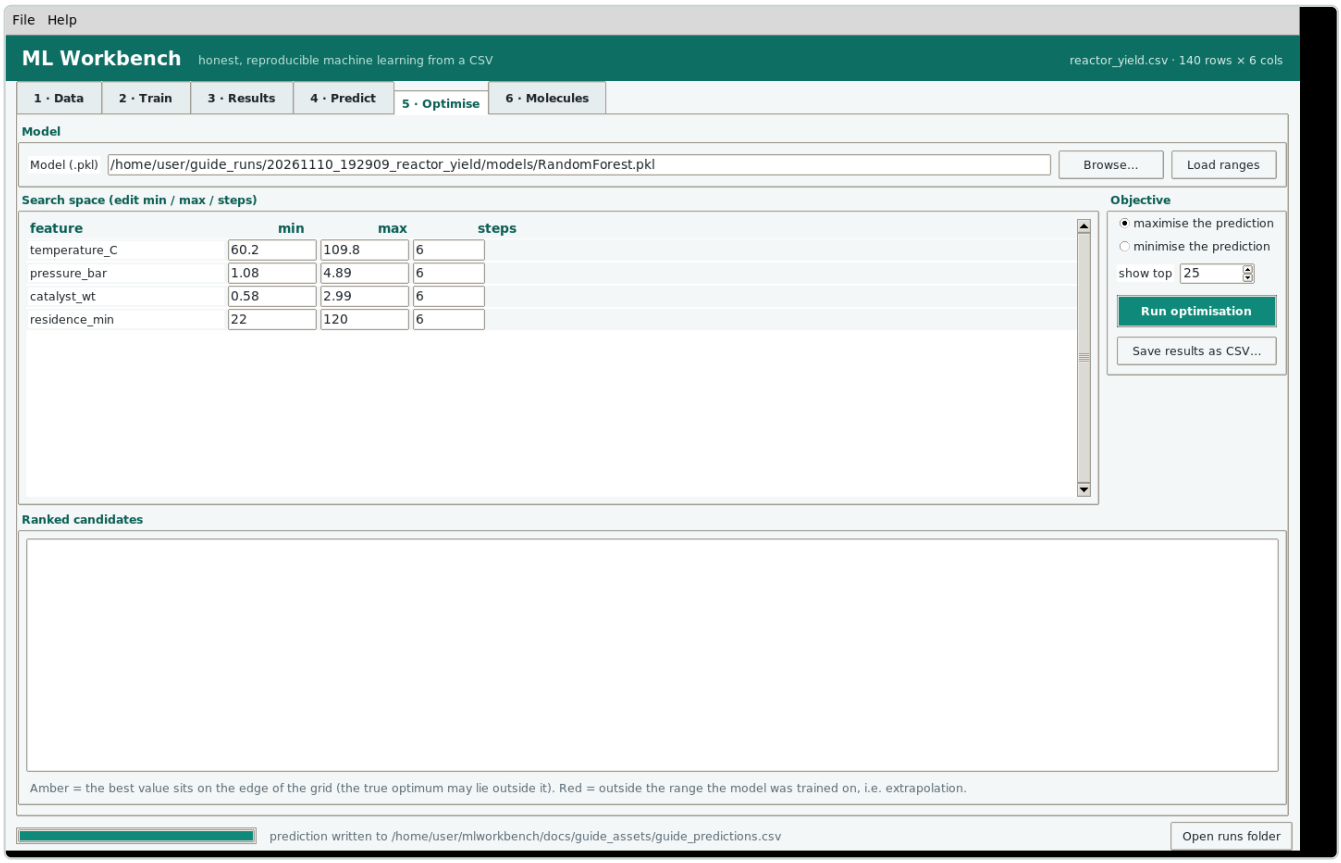


Figure 17.1 — Ranges loaded from the RandomForest model. Each numeric feature shows the training min and max, so the default grid is exactly the space you have evidence for. The objective panel on the right holds the maximise/minimise choice, the "show top" count and the two buttons.

17.3 What the run produces

Four numeric features × 6 steps = 7 values each = **7^4 = 2,401 candidates**. With the default training ranges, sorted by predicted yield, the top of the table reads:

temperature_C	pressure_bar	catalyst_wt	residence_min	predicted_yield_pct	_at_grid_boundary	_inside_training_range
93.27	4.89	2.59	103.7	69.53	pressure_bar	yes
85.00	4.89	2.59	103.7	69.48	pressure_bar	yes
93.27	4.89	2.59	120.0	69.24	pressure_bar, residence_min	yes
93.27	4.89	2.99	103.7	69.05	pressure_bar, catalyst_wt	yes
60.20	1.08	0.58	71.0	35.41	—	yes

Last row = the worst candidate, for scale: the model believes yield runs from about 35 % to about 70 % across the space you have explored.

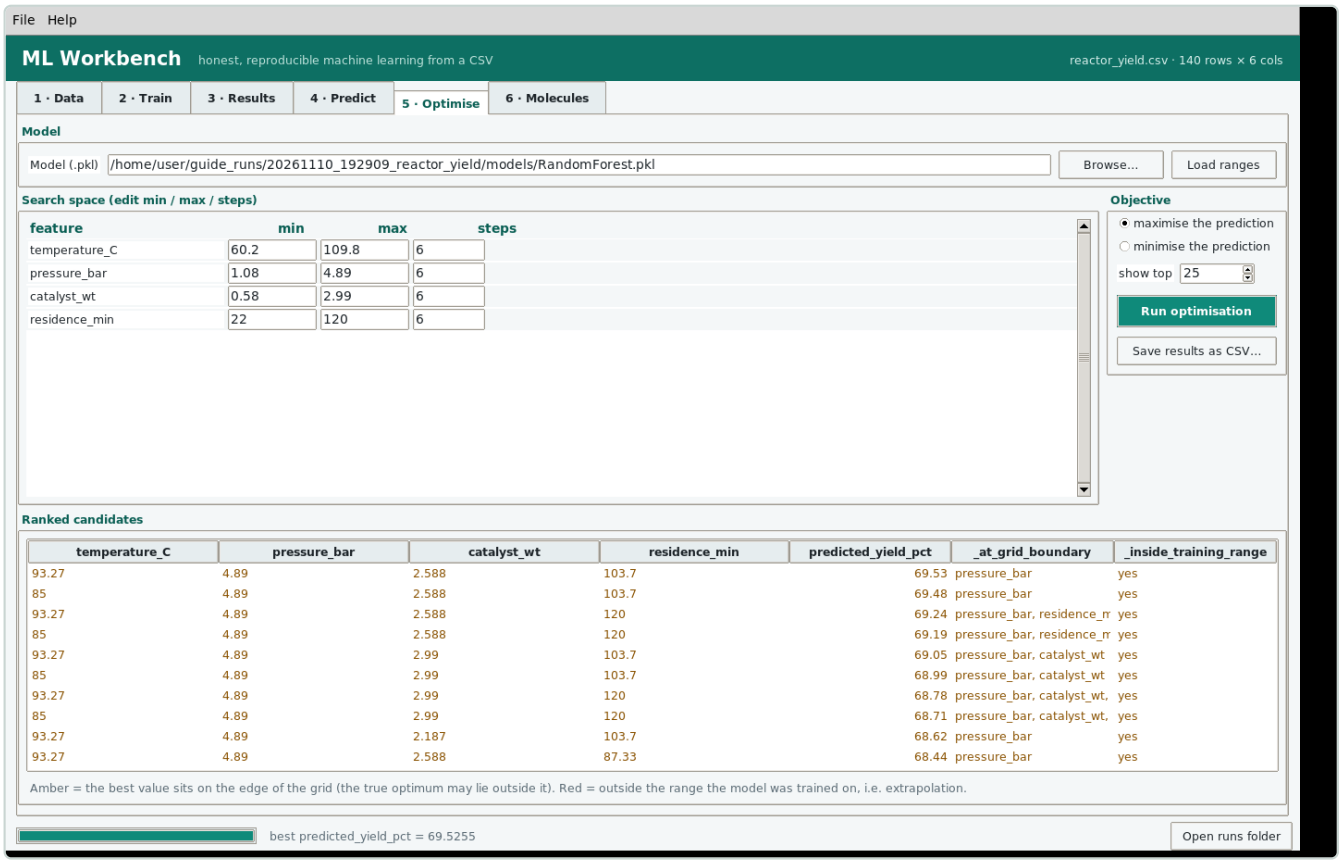


Figure 17.2 — Ranked candidates. Amber rows have at least one value sitting on the edge of the grid; red rows are outside the training range entirely. The legend under the table repeats this, because it is the difference between a plan and a fantasy.

17.4 How to read the boundary warning

Every one of the top rows has `pressure_bar` at 4.89 — the maximum in the dataset. That is not a coincidence, and it is not a recommendation to run at 4.89 bar. It means:

- yield rises with pressure across the whole observed range;
- the model has **no evidence** about what happens above 4.89 bar;
- so the honest conclusion is "push pressure as high as your equipment allows and re-measure", not "4.89 bar is optimal".

> WATCH OUT
>
> An optimum at the edge of the search space almost always means the true optimum is outside it. The correct next step is a small, deliberate experiment at the boundary — 5.2 and 5.6 bar, say — not a production change based on a model's extrapolation. The program tags those rows in amber for exactly this reason.

17.5 Other features held fixed

Features you do not put in the grid are not varied: numeric ones are held at the midpoint of their training range, categorical ones at their first known level. If a variable you care about is missing from the table, it is because the model did not have it as a feature — check the artefacts panel on the Results tab.

17.6 Minimising

Switch the objective to **minimise** for costs, impurity, cycle time or defects. The mechanics are identical; only the sort order changes. A worked example: train on cycle time, then minimise to find the fastest conditions the model believes are achievable.

TRY IT

Set `pressure_bar`'s max to 6.0 and re-run. The best predicted yield rises above 69.5 % and the boundary flag stays on pressure — but now the row is **red**, because 6.0 bar was never seen in training. Two numbers that differ by 0.2 % in the table mean completely different things.

TUTORIAL 6 · CHAPTER 18

Molecules: SMILES to features, then a model

Goal: convert a list of molecules into numbers a model can use, then train a predictive chemistry model on the result — and know when it is safe to use on a new molecule.

18.1 Why chemistry needs this tab

A model cannot consume a drawing or a structure; it consumes numbers. A **SMILES** string is a compact text representation of a molecule — `CCO` is ethanol, `c1ccccc1` is benzene. The Molecules tab turns each SMILES into either a short row of physically meaningful **descriptors** (molecular weight, logP, polar surface area, ring counts…) or a long **fingerprint** (a bit-vector describing which substructures are present). Either output is a CSV you can then train on with the other tabs.

18.2 The controls

Control	Meaning
CSV with SMILES	The input table. Any other columns (name, measured property) are carried through untouched.
SMILES column	Which column holds the strings. Detected automatically when a column is called `SMILES` or `smiles`.
descriptors	15 numeric columns: `MolWt`, `MolLogP`, `TPSA`, `NumHDonors`, `NumHAcceptors`, `NumRotatableBonds`, `RingCount`, `FractionCSP3`, `HeavyAtoms`, `qed`, `NumAromaticRings`, `MaxAbsEStateIndex`, `MinAbsEStateIndex`, `BalabanJ`, `BertzCT` — plus a `canonical_smiles` column. Compact and interpretable: good for small datasets.
Morgan fingerprint	A fixed number of bits (1024 by default) recording which substructures exist within `radius` bonds. The workhorse of modern QSAR. `radius 2` is the standard "ECFP4" setting.
RDKit fingerprint	Path-based bits up to `maxPath` bonds (default 7). Different bias from Morgan; useful as a comparison.
standardise (largest fragment)	Salts and mixtures — `[Na+].[Cl-]`, `CCO.CC(=O)O` — are reduced to their largest fragment before featurisation, and the affected rows are reported. Turn it off only if the salt form is genuinely part of the question.
Check applicability domain…	Compares each query molecule to a training set and reports the maximum Tanimoto similarity plus an `in_domain` flag. This is the chemistry equivalent of the extrapolation warning.

1. Load `molecules.csv` — 34 molecules with `name`, `SMILES` and a measured `logS`.
2. Keep **descriptors** selected and **standardise** ticked. Press **Generate features**.
3. Read the output panel: how many molecules, how many columns, where the file was written.
4. Open the written CSV and confirm each input row gained descriptor columns and a `canonical_smiles`.

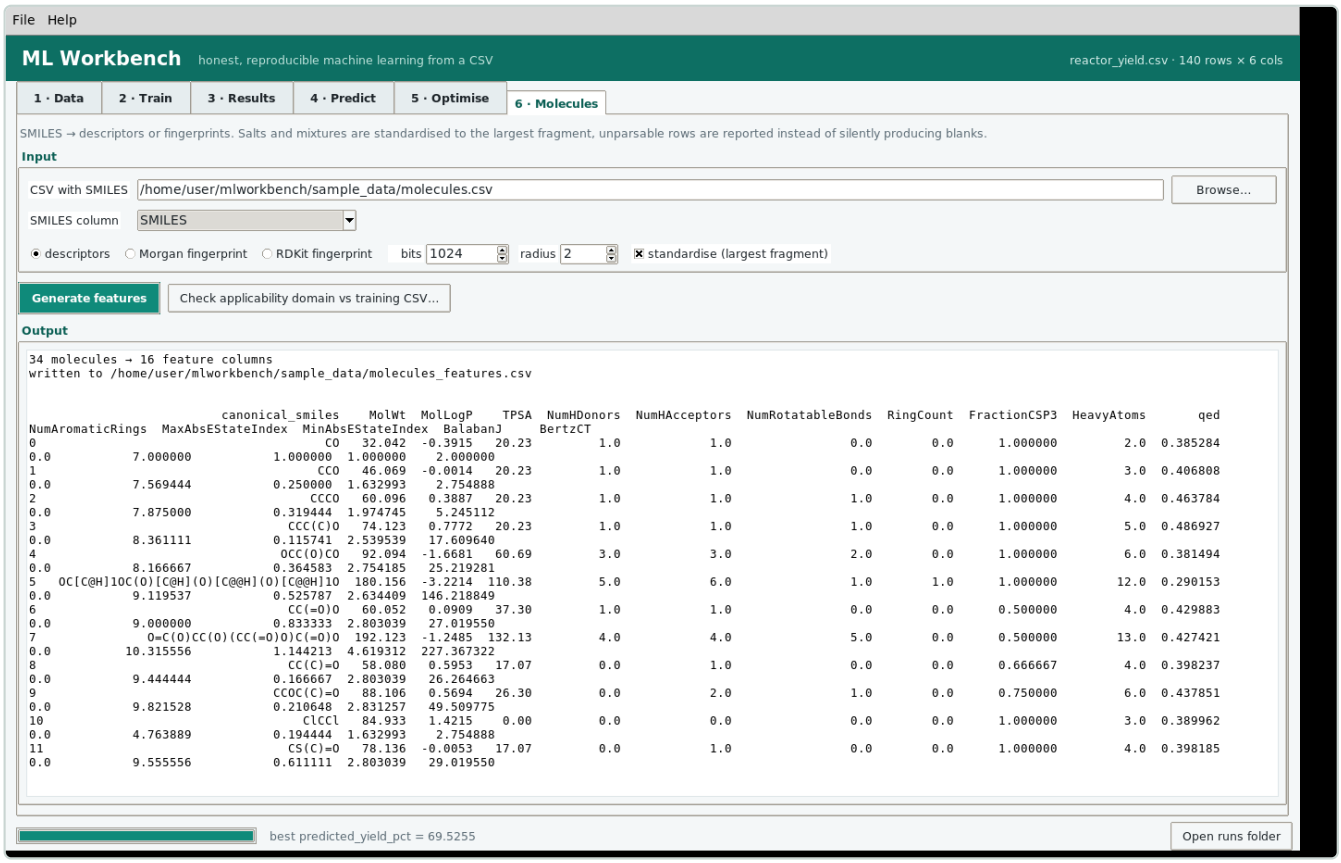


Figure 18.1 — Descriptors generated. The report says 34 molecules → 16 feature columns; the preview shows the canonical SMILES and the descriptor values. Methanol (`CO`) has MolWt 32.04 and logP −0.391; benzene has six aromatic carbons and `NumAromaticRings` 1.

WHAT YOU SHOULD SEE

```
34 molecules -> 16 feature columns
written to ...\molecules_features.csv
```

The 16 columns are `canonical_smiles` plus the 15 descriptors. Descriptors that cannot be computed by the installed RDKit build are dropped and named in the note rather than left as empty columns; unparsable SMILES produce an empty row and a warning naming the row numbers.

18.3 Cleaning up before you train

The output contains `name` and `canonical_smiles` as text columns. Before training, open the CSV in the app and **untick** both: a name is an identifier and a SMILES string is a unique per-row fingerprint — the fastest way to make a chemistry model that scores 1.00 and predicts nothing.

18.4 A complete QSAR model on 34 molecules

1. Load `molecules_features.csv` on the Data tab.
2. Set the target to `logS` (aqueous solubility, a real measured property).
3. Untick `name` and `canonical_smiles`.
4. Train with **Ridge**, **RandomForest** and **GradientBoosting**, 5 folds, holdout ticked. With 34 rows expect a large spread — the health check will warn you.

5. Read the result as a screening tool, not a predictor: with this little data the model can rank molecules by likely solubility, but not tell you a number you should trust to two decimals.

> WATCH OUT
>
> Chemistry datasets are usually small and clustered by scaffold. Two molecules in different folds that share a scaffold make the model look better than it is. If you have a scaffold or series identifier, use the **group** CV scheme with that column (chapter 20) — that is the honest way to ask "will this work on a new chemical series?".

18.5 The applicability domain, in practice

Before predicting solubility for a new compound, press **Check applicability domain vs training CSV…** and point it at your training molecules. The output has one row per query with `max_similarity`, `nearest_index` and `in_domain` (similarity ≥ 0.4 to at least one training molecule). A molecule outside the domain is one the model has never seen anything like; its prediction is a guess dressed as a number.

TUTORIAL 7 · CHAPTER 19

Classification, end to end

Goal: run the complete classification workflow on `reactor_grade.csv`, including the class-balancing decision, and interpret the two metrics that actually matter.

19.1 The dataset

140 batches, 8 columns: the same four process variables, three text columns (`reactor` R1/R2/R3, `solvent` water/ethanol/toluene, `shift` day/night) and the target `grade` with three classes:

Class	Rows	Share	Practical meaning
A	49	35 %	good
B	72	51 %	the common outcome — the majority class
C	19	14 %	the minority — usually the one that costs money

19.2 Run it

1. Load `reactor_grade.csv`, set the target to `grade`. The task line should read `classification`; the health check reports the class balance.
2. Tick the six real features and **untick** `shift` — try it included and excluded and compare; if shift is a proxy for the operator or the day, it will not exist in the future you are predicting.
3. Train tab: **none**, then tick **Logistic** and **RandomForest**.
4. Tick **Balance classes (class_weight)**. This matters with a 14 % minority class.
5. Keep 5 folds, holdout 0.2, seed 77, search budget 12. Run.

WHAT YOU SHOULD SEE

```
Logistic      accuracy 0.7862   balanced acc 0.7654   F1 macro 0.7500
              holdout: acc 0.7857  bal.acc 0.8476  F1 0.7781  ROC AUC 0.9357
RandomForest  accuracy 0.7324   balanced acc 0.6824   F1 macro 0.6884
              holdout: acc 0.8571  bal.acc 0.7762  F1 0.8111  ROC AUC 0.9104
BASELINE      accuracy 0.5178   F1 macro 0.2274      holdout acc 0.5000
```

The recommendation picks **Logistic**: it wins on the primary metric (macro F1), and it is the simplest model on the table.

19.3 Why F1 and not accuracy

The baseline already scores 0.52 accuracy by always answering “B”. Both models beat that, but the interesting question is whether they can identify a C. Macro F1 averages per-class performance, so a model that never predicts C is punished regardless of its accuracy. Logistic regression reaches 0.75; the forest reaches 0.69 on cross-validation but does better on the 28 holdout rows (0.81). With this little data, that disagreement is itself the finding: the two models are close, and the choice should rest on interpretability unless you have more data.

19.4 Reading the confusion matrix

For classification runs the run folder contains `plots/confusion_matrix.png`: true classes down the side, predicted classes across the top, counts in the cells. Read the diagonal (agreeing cases) and then the off-diagonal cells, which are the specific mistakes:

- **C predicted as B** — the expensive error: a bad batch passes.
- **A predicted as B** — a good batch is downgraded: waste, but safe.
- **B predicted as A** — the mirror of the first case.

Which off-diagonal cell you care about is a business decision, and it is the reason the program keeps the whole matrix rather than a single score.

19.5 Using the classifier

Predict with a CSV of new conditions that includes the same text columns. The output contains `predicted_grade` plus `proba_A`, `proba_B` and `proba_C`. A practical rule: act on the predicted class only when its probability is clearly ahead; a row with A 0.42 / B 0.40 / C 0.18 is a “send to the lab” case, not an “A”.

> TIP
>
> If a class is very rare (under ~10 rows), collect more of it before trusting any model. If you cannot, use the classifiers that handle imbalance natively — RandomForest, ExtraTrees, GradientBoosting — with **Balance classes** ticked, and report balanced accuracy and F1, never accuracy alone.

> TRY IT
>
> Re-run the classification without ticking **Balance classes** and compare class C’s recall in the confusion matrix. The difference is what class weighting buys you: usually a few more Cs caught, at the price of a few more false alarms.

TUTORIAL 8 · CHAPTER 20

Time series and grouped data

Goal: two validation schemes that stop you from accidentally predicting the past using the future, or recognising a batch you have already seen.

20.1 Grouped data — several rows per unit

If your table has several measurements per batch, per patient, per wafer or per reactor campaign, the rows are not independent. Plain `kfold` will happily put three rows of the same batch in the training folds and the fourth in the scoring fold, and the model will score brilliantly by recognising the batch rather than the physics.

1. Make sure you have an identifier column that is constant within a unit and different between units — `batch_id`, `patient_id`, `campaign`.
2. On the Train tab set **CV scheme** to `group`; the **Group column** dropdown becomes active. Choose your identifier.
3. Train as usual. The folds are now cut by group: an entire batch is either in training or in scoring, never split.
4. Compare with the `kfold` score. A large drop is the size of the illusion you were previously reporting.

> WATCH OUT
>
> Do not use the group identifier as a feature while using it for grouping — that reintroduces the recognition problem. The program flags unique-per-row columns, but a batch ID with several rows per batch is not unique, so it will not be flagged. Untick it yourself.

20.2 Time series — the future must be predicted from the past

With chronological data, random folds leak: the model learns from December to score November. The `timeseries` scheme uses expanding windows instead — train on the first block, score on the next, then extend.

```
fold 1:  [train] [score]
fold 2:  [train] [train] [score]
fold 3:  [train] [train] [train] [score]
```

Lag features are the other half of the job: yesterday's value is usually the best predictor of today's. The **Time-series features** box on the Train tab does this for you.

20.3 A worked example: daily output

The guide ships a synthetic factory dataset, `docs/guide_assets/daily_output.csv`: 360 rows, two production lines, 180 consecutive days, columns `date`, `line`, `operators`, `temp_C`, `output_t`. It has a slow upward trend, a weekly cycle and weekend step, and noise — exactly the shape of most real production series.

Run it from the command line (chapter 21 explains the syntax):

```
MLWorkbench.exe train daily_output.csv --target output_t ^
    --lag 1,7 --roll 7 --group line --sort-by date ^
    --cv timeseries --models Ridge,RandomForest --folds 4 --search 8 --holdout 0.2
```

WHAT YOU SHOULD SEE

```
Lag features: 360 -> 346 rows (rows without history dropped)
Data: 346 rows, 7 features (6 numeric, 1 categorical), target 'output_t' [regression]
CV: timeseries, 4 folds, seed 77, holdout 20%
Baseline r2: -0.0188
→ Ridge        r2 = 0.8773 (+/- 0.0167)   MAE 4.66 t
→ RandomForest r2 = 0.8625 (+/- 0.0293)   MAE 4.81 t
Recommended: Ridge
Gain over baseline: +0.8961
Holdout: r2 0.8983, MAE 3.78 t, MAPE 3.1 %
Feature importance: output_t_lag7=+1.054, output_t_lag1=+0.031,
line=+0.009, ..._roll7_mean=+0.002
```

20.4 Reading that result

- **346 rows, not 360:** the first 7 days of each line have no history, so they were dropped. That is the documented cost of lagging, not an error.
- **lag7 dominates** — the model found the weekly cycle. In your own data, whichever lag dominates tells you the period of the cycle, which is often the most valuable single insight on the screen.
- **Ridge beat RandomForest.** With a strong, smooth, additive seasonal signal, a linear model is the right tool; trees need more data to approximate a smooth trend. This is the normal outcome and the reason the recommendation prefers simplicity.
- **Holdout R² 0.898, MAE 3.78 t, MAPE 3.1 %** — 70 future days, never used for tuning. That is the number to quote for planning.

WATCH OUT

The `--sort-by date` flag sorts the rows before lagging. Without it the lags are built in whatever order the file happened to be in, and the whole exercise is meaningless. Also keep the `--group line` here: without it, the first rows of line L2 would be lagged with the last rows of L1.

20.5 In the GUI instead

The GUI exposes the same feature set without the group/sort options: put the lag numbers in the **Lags** box on the Train tab and choose `timeseries` as the CV scheme. For multi-series files with a date column, the command line is the better tool — that is the honest division of labour between the two interfaces.

TUTORIAL 9 · CHAPTER 21

The command line, command by command

Goal: do everything the GUI does — and a few things it does not — from a script, so that a monthly report can run unattended.

21.1 Which mode am I in?

One executable, two personalities, decided by its arguments:

You run	You get
`MLWorkbench.exe`	the GUI
`MLWorkbench.exe C:\data\file.csv`	the GUI, with that file already loaded (also works via Open with)
`MLWorkbench.exe --help`	the command list, in the console
`MLWorkbench.exe train ...`	the CLI — output goes to the console you launched it from
`MLWorkbench.exe --selftest C:\temp\check`	a headless end-to-end verification; writes `selftest.json`

Open a console first (**Win+R** → `cmd`), then either `cd` to the folder with the exe or use the full path. Exit codes: **0** success, **2** a data/schema problem (the message says which), **1** an unexpected error, **130** interrupted with Ctrl+C.

> TIP
>
> Typing `MLWorkbench.exe` in full is tedious. Make a shortcut or add the folder to your `PATH`, then use `MLWorkbench train ...` from anywhere. The command name is the exe name; if you prefer the short form used in the source distribution, `python -m mlw` does the same thing.

21.2 `audit` — check a file before you model it

```
MLWorkbench.exe audit data.csv --target grade
```

Prints the file's shape, encoding and separator, then every health-check finding with its severity and hint, then the inferred task. This is the fastest way to answer "is this file going to work?" without opening the window — and the right first command in any automated pipeline.

```
messy_demo.csv: 12 rows x 9 cols, encoding=utf-8-sig, sep=','
[WARNING] small-n            Only 12 rows. Cross-validation scores will have a wide spread.
          -> Treat differences under ~0.05 as ties and prefer the simpler model.
[WARNING] id-columns         Column(s) that look like identifiers: run_id
          -> Unique-per-row columns are leakage. Do not use them as features.
[ERROR  ] target-missing     Target 'grade' has 1 missing value(s).
          -> Rows without a target cannot be trained on. Use a predict file instead.
...
Inferred task for 'grade': classification
```

21.3 `train` — the whole training tab in one line

```
MLWorkbench.exe train reactor_yield.csv --target yield_pct --holdout 0.2 --search 12 ^
    --models Linear,Ridge,RandomForest,GradientBoosting
```

Flag	Default	Meaning
`--target`	required	Column to predict.
`--features`	all other columns	Comma-separated allow-list. Use it to exclude leakage explicitly — safer than trusting a default.
`--task`	`auto`	`auto`, `regression`, `classification`.
`--models`	`auto`	`auto` (the six recommended), `all`, or a comma-separated list of names from chapter 24.
`--folds`	5	Cross-validation folds.
`--cv`	`kfold`	`kfold` | `group` | `timeseries`.
`--group`	—	Group column for `--cv group`, and the series separator for lags.
`--holdout`	0.0	Fraction locked away (0–0.5). Set 0.2 for a final honest number.
`--seed`	77	Random seed.
`--search`	0	Randomised hyperparameter trials per model; 0 = library defaults.
`--balance`	off	Class weighting for imbalanced classification.
`--lag`, `--roll`, `--sort-by`	—	Time-series features: lags (`1,2,7`), rolling window sizes (`7,14`), and the date column to sort by first.
`--out`	`runs`	Folder for run folders.
`--json`	—	Also write a JSON summary of the result here (handy for dashboards).
`--save-all`	off	Save every candidate model, not only the chosen one.

It prints the comparison table, the recommendation, and the paths to the model and the report.

21.4 `predict` — apply a saved model

```
MLWorkbench.exe predict runs\20261110_192909_reactor_yield\models\RandomForest.pkl ^
    new_batches.csv -o predictions.csv
```

```
10 row(s) written to predictions.csv
predicted column: predicted_yield_pct
WARNING: 2 row(s) of 'temperature_C' outside training range 60.2…109.8
```

Exit code 2 if required columns are missing, 0 otherwise (warnings do not fail the run — that is deliberate, so a nightly job is not blocked by two odd rows).

21.5 `optimise` — rank candidates

```
MLWorkbench.exe optimise RandomForest.pkl -r temperature_C:60:110:10 -r pressure_bar:1:5:8 -o
grid.csv
```

Ranges are `name:min:max:steps` and may be repeated. Categorical features use `-l name:v1|v2|v3`. Add `--minimise` to sort ascending, `--top 50` to change how many rows are printed. The output CSV always contains the **whole** grid with the `predicted_…`, `_at_grid_boundary` and `_inside_training_range` columns.

```
         105.0  4.5  1.785  71.0  61.539291                            True
          80.0  5.0  1.785  71.0  61.517242   pressure_bar             False
...
99 candidates written to grid.csv
```

Note how 5.0 bar is marked `False` in `_inside_training_range`: the model never saw above 4.89 bar, so that candidate is extrapolation even though it looks ordinary.

21.6 `smiles` — chemistry features from the console

```
MLWorkbench.exe smiles molecules.csv --column SMILES --mode descriptors -o features.csv
MLWorkbench.exe smiles molecules.csv --mode morgan --bits 2048 --radius 3 -o fp.csv
MLWorkbench.exe smiles new_molecules.csv --ad-against molecules.csv -o new_features.csv
```

Flag	Meaning
`--mode`	`descriptors` (15 numbers), `morgan`, `rdkit`.
`--bits`, `--radius`, `--max-path`	Fingerprint size, Morgan radius, RDKit path length.
`--no-standardise`	Keep salts and mixtures intact instead of reducing to the largest fragment.
`--ad-against`	CSV of training SMILES; writes an extra `*_applicability_domain.csv` with similarity and `in_domain`.

21.7 Scripting: three patterns worth stealing

One model per target, in a loop. Windows batch, one line per target:

```
for %T in (yield_pct purity_pct) do (
  MLWorkbench.exe train reactor_yield.csv --target %T --holdout 0.2 --search 12 ^
      --models Linear,Ridge,RandomForest --out runs_%T --json runs_%T\summary.json
)
```

A nightly scorecard. Audit, then train, then predict, and stop if the audit fails:

```
MLWorkbench.exe audit new_data.csv --target grade || exit /b 1
MLWorkbench.exe train new_data.csv --target grade --cv group --group batch_id ^
    --balance --holdout 0.2 --out runs\grade
MLWorkbench.exe predict runs\grade\latest\models\RandomForest.pkl to_score.csv -o scored.csv
```

Copy the latest model forward. The run folder is timestamped, so pick the newest by name: the folders sort chronologically, which is what makes a simple “latest” reference possible.

> **WATCH OUT**
>
> The command line has no confirmation prompts and no undo. Point `--out` at a dedicated folder in automated jobs so a run never overwrites something you meant to keep.

TUTORIAL 10 · CHAPTER 22

Fitting it into your working day

Goal: the habits that separate a model you can defend from a number you happened to produce on a Tuesday.

22.1 A working routine

1. **Write down the question before you open the program.** "Predict yield from the four controllable settings, for planning next month's campaign." A vague question produces a model nobody uses.
2. **Audit first.** Data tab → read the health check → fix the file, not the model.
3. **Exclude leakage deliberately.** For each column ask "would I know this at prediction time?". Write the excluded columns in your notes; the next person will ask.
4. **Run the recommended preset once.** Do not tune anything until you have seen the first honest table.
5. **Read the recommendation and the spread before the score.** If the best model is within noise of the baseline, the honest deliverable is "no usable model from this data".
6. **Predict on the rows you actually care about** and check the warnings. Extrapolated rows are not predictions.
7. **Archive the run folder** next to the decision it informed — not in a chat message.

22.2 Naming so that future-you can find things

Thing	Recommendation
Run folders	Leave them alone. They are timestamped by the program and sort chronologically; renaming breaks that order without adding information.
Models you deploy	Copy the `.pkl` to a folder named after its purpose and date — `yield_model_2026-11.pkl` — and keep the run folder it came from.
Prediction outputs	Include the question: `campaign_2026-12_candidates.csv` beats `predictions(3).csv`.
Notes	One text file per decision: the question, the run folder, the excluded columns, the number you quoted, who asked.

22.3 When to retrain

- **New data changes the process** — new equipment, new catalyst supplier, new product grade.
- **Predictions drift**: score a recent period where you know the answer and compare the error with the reported MAE. If the real error is much larger, retrain.
- **Quarterly, as hygiene**, on the accumulated data. More data usually beats more tuning.

22.4 Sharing results without over-claiming

A defensible one-paragraph summary contains five things. Here is a worked example from the tutorial run:

> EXAMPLE SUMMARY
>
> “Random forest predicts reactor yield from temperature, pressure, catalyst charge and residence time with R^2 0.72 on cross-validation (spread ±0.06) and MAE 2.6 percentage points on a holdout of 28 experiments never used for tuning. A model that always predicts the mean scores R^2 −0.14, so the gain is real. Temperature dominates (importance 1.17 versus 0.31 for pressure). Purity was excluded because it is measured after the run. Full settings, seed and data hash are in the run folder `20261110_192909_reactor_yield`.”

What that paragraph does not do: call the model “accurate”, promise a percentage improvement, or quote the training score. Those three are how analyses lose credibility in review.

22.5 What to say when someone asks for 95 %

Ask what decision the number supports. If the model explains 72 % of the variation and the misses average 2.6 points, the useful question is whether a 2.6-point error changes the decision — often it does not, and the model is already good enough. If it does, the answer is more data or a tighter measurement, not more clicking.

22.6 A review checklist

- ☐ The target is the thing that actually matters, and it was measured consistently.
- ☐ No feature would be unavailable at prediction time.
- ☐ Identifier columns are excluded; group/timeseries CV used where the rows are not independent.
- ☐ The baseline is on the table and the gain over it is stated.
- ☐ The spread is quoted next to every average.
- ☐ The holdout score was looked at once, after the model was chosen.
- ☐ Predictions outside the training range are flagged and not used for decisions.
- ☐ The run folder — with `run.json`, the seed and the data hash — is archived.

PART IV

Reference

Everything you look up after the first week: the field-by-field reference for each tab, the model catalogue, the run-folder file list, the metric definitions, a troubleshooting list built from the failures people actually hit, a data-preparation checklist and a glossary.

IN THIS PART

23 · Field-by-field reference for every tab

24 · Model catalogue

25 · Run folder and file reference

26 · Metric reference

27 · Troubleshooting: 24 problems and their fixes

28 · The data-preparation checklist

29 · Glossary

CHAPTER 23

Field-by-field reference for every tab

23.1 Data tab

Control	Type	Notes
Open CSV…	button	Ctrl+O. Detects encoding (UTF-8, UTF-8-BOM, Windows-1252, Shift-JIS, Latin-1) and separator (comma, semicolon, tab).
Preview	table	First 200 rows. Read-only. Mouse wheel scrolls vertically.
Target column	dropdown	Every column is offered. Selecting one unticks it as a feature.
Detected task	label	`regression` / `classification` plus the target range. Informational — the Train tab decides.
Features / all / none	checkboxes + buttons	The only place leakage is prevented. Count shown at the right.
Data health check	list	Severity + finding. Click a row for its hint. Refreshes when the target changes.

23.2 Train tab

Control	Type / range	Notes
recommended / all / none	buttons	Presets for the model tick list.
Model list	checkboxes	Contents depend on the task. ★ = in the recommended preset.
Task	`auto` · `regression` · `classification`	Overrides the automatic inference.
Folds	2–20, default 5	Clamped automatically for tiny datasets.
CV scheme	`kfold` · `group` · `timeseries`	See chapter 5 and chapter 20.
Group column	dropdown	Active only for `group`.
Holdout fraction	checkbox + 0.05–0.50	Ticked with 0.2 by default. Untick to skip the holdout entirely.
Random seed	0–99999, default 77	Same seed = same result.
Search budget	0–200, default 12	0 = library defaults (fast). 12–30 recommended.
Balance classes	checkbox	Classification only; ignored by models without class-weight support.
Save every model	checkbox	Writes a `.pkl` per candidate.
Lags	text, e.g. `1,2,7`	Adds lag and rolling-mean features; drops rows without history.
Run training	button	Disabled while a run is in progress.
Log	text panel	Timestamped transcript of the run. Kept until the app closes.

23.3 Results tab

Control	Notes
Recommendation banner	One sentence naming the model and the justification, including the comparison with the baseline.
Model comparison table	Sortable; each row is a model, plus the baseline. Click a row for hyperparameters and per-fold scores.
What drives the prediction	Permutation importance, sorted descending; negative values mean the feature hurt.
Artefacts panel	Lists the files written, then four buttons: open folder, open report, view plot, send model to Predict.
Status bar	`done · recommended <Model>` when a run completes.

23.4 Predict tab

Control	Notes
Model (.pkl)	Any compatible model file. Filled automatically after training or by the Results button.
New data (CSV)	Needs the model's feature columns; extra columns ignored; missing columns reported by name.
Output CSV	Blank → `<input>_predictions.csv` beside the input file.
Validate & predict	Runs the schema check, writes the file, fills the report and the 100-row preview.
Report panel	Green = success line; amber = warnings (unseen categories, imputation, coercion); red = extrapolation detail or failure.

23.5 Optimise tab

Control	Notes
Model (.pkl) + Browse	The model to interrogate. Must contain the features you intend to vary.
Load ranges	Pre-fills min/max from the training data, 6 steps, and the known levels for categorical features.
Search space rows	`feature | min | max | steps`. Steps is the number of intervals (6 → 7 values). Categorical rows take `v1|v2|v3`.
maximise / minimise	Sort direction of the ranked output.
show top	5–200 rows displayed (the CSV always holds everything).
Run optimisation / Save results as CSV…	Evaluate the grid; then write it out.
Ranked candidates	Amber = value at the edge of the grid; red = outside the training range.

23.6 Molecules tab

Control	Notes
CSV with SMILES / Browse	Input table; other columns are carried through to the output.
SMILES column	Auto-detected when named `SMILES` / `smiles`.
descriptors · Morgan · RDKit	Output type. Descriptors = 15 columns; fingerprints = `bits` columns named `fp0000…`.
bits / radius	Fingerprint length (default 1024) and Morgan radius (default 2).
standardise (largest fragment)	Reduces salts and mixtures to the largest fragment; reported per row.
Generate features	Writes `<input>_features.csv` beside the input unless you are prompted otherwise.
Check applicability domain…	Compares against a training CSV; writes `max_similarity`, `nearest_index`, `in_domain` (≥ 0.4).

23.7 Menus and global controls

Item	Action
File → Open data CSV… (Ctrl+O)	Same as the Data tab button.
File → Open run folder	Explorer at the runs directory.
File → Exit	Closes the window. Nothing is saved that was not already written.
Help → Quick guide	One-screen summary of the six tabs.
Help → About	Version, Python and scikit-learn versions, and the list of design guarantees.
Open runs folder (status bar)	Same as File → Open run folder.

CHAPTER 24

Model catalogue

What each model family is, when it tends to win, and what the program tunes when you set a search budget. “Scale” means the features are standardised before fitting — you never do it yourself, it happens inside the model.

24.1 Regression models

Model	Cplx	Scale	What it is	Use it when
Linear	0	no	Straight-line fit.	The reference point. If a straight line is nearly as good as the forest, use the line.
Ridge	1	yes	Linear with a penalty that shrinks unstable coefficients.	Correlated features (all four process variables usually are). Almost always ≥ Linear.
Lasso	1	yes	Linear with a penalty that can zero coefficients.	You suspect only a few features matter and want them named.
ElasticNet	1	yes	Ridge + Lasso penalty.	Many correlated features, some irrelevant.
PLS	2	yes	Projects features onto a few latent components.	Many highly correlated features, few rows — chemometrics standard.
LinearSupportVector	2	yes	Linear model fitted to a margin.	Many rows, robustness to outliers matters.
SupportVector	3	yes	Non-linear kernel regression.	Smooth non-linear response, ≲ 10 000 rows — it scales poorly above that.
KNeighbors	2	yes	Average of the nearest training rows.	Very non-linear, dense data. Useless for extrapolation by construction.
DecisionTree	3	no	One tree of if/then splits.	You need an explainable rule. Expect a worse score than the forest.
RandomForest	4	no	Many trees, averaged.	**The default choice.** Hard to break, handles mixed column types, needs little tuning.
ExtraTrees	4	no	Randomised forest.	Fast, slightly lower variance than the forest; try it when the forest is close to the top.
GradientBoosting	5	no	Trees fitted sequentially to the residual.	Usually the accuracy leader on tabular data with a few thousand rows; slower and easier to overfit.
HistGradientBoosting	5	no	Boosted trees with histogram binning.	Large datasets; 10–100× faster than GradientBoosting, comparable accuracy.
MLP	5	yes	Small neural network.	Large datasets with complex interactions. Rarely wins on a 140-row table.
XGBoost	6	no	Extreme gradient boosting.	Only appears if the `xgboost` package is installed — it is not in the shipped exe. Install it for large tabular problems.

24.2 Classification models

Model	Cplx	Scale	Bal.	What it is	Use it when
Logistic	1	yes	yes	Linear decision boundary with probabilities.	The interpretable default; coefficients become “log-odds per unit”.
LinearDiscriminantAnalysis	1	yes	no	Gaussian classes with a shared covariance.	Few rows, roughly Gaussian features; very fast.
GaussianNaiveBayes	1	no	no	Independent-Gaussian assumption.	A very fast, very biased baseline. Useful for text-like features.
KNeighbors	2	yes	no	Majority vote of nearest rows.	Local structure, small datasets.
SupportVector	3	yes	yes	Kernel boundary with probabilities.	Clear margins, moderate size. Slow on big data.
LinearSupportVector	2	yes	yes	Linear margin classifier.	Many rows, fast; **no probability output**.
DecisionTree	3	no	yes	Single tree.	You must show the rules.
RandomForest	4	no	yes	Averaged trees.	**The default choice** for classification too.
ExtraTrees	4	no	yes	Randomised forest.	Fast comparison to the forest.
GradientBoosting	5	no	no	Boosted trees.	Accuracy leader on medium data.
HistGradientBoosting	5	no	no	Histogram-boosted trees.	Large data, fast.
QuadraticDiscriminantAnalysis	2	no	no	Gaussian classes, class-specific covariance.	Few rows per class, quadratic boundaries.
MultilayerPerceptron	5	yes	no	Small neural network.	Large datasets with complex interactions.
XGBoost	6	no	no	Extreme gradient boosting.	Only if the package is installed; not in the shipped exe.

24.3 What the search budget tunes

Model	Hyperparameters explored
Ridge / Lasso	`alpha` (penalty strength)
ElasticNet	`alpha`, `l1_ratio`
PLS	`n_components`
SupportVector / LinearSupportVector	`C`, `gamma` (kernel), `epsilon` (regression), `kernel` (classification)
KNeighbors	`n_neighbors`, `weights`
DecisionTree	`max_depth`, `min_samples_leaf`, `criterion`
RandomForest / ExtraTrees	`n_estimators`, `max_depth`, `min_samples_leaf`, `max_features`
GradientBoosting	`n_estimators`, `learning_rate`, `max_depth`, `subsample`
HistGradientBoosting	`max_iter`, `learning_rate`, `max_leaf_nodes`
MLP	`hidden_layer_sizes`, `alpha`
Logistic	`C`
LinearDiscriminantAnalysis	`solver`
GaussianNaiveBayes	`var_smoothing`
QuadraticDiscriminantAnalysis	`reg_param`
XGBoost	`n_estimators`, `learning_rate`, `max_depth`, `subsample`

The chosen values are printed in the log, stored in `run.json`, shown in the model-detail dialog, and embedded in the saved model — so a prediction made later uses exactly the tuned settings.

CHAPTER 25

Run folder and file reference

```
runs/
└── 20261110_192909_reactor_yield/
    ├── run.json
    ├── report.md
    ├── metrics.csv
    ├── feature_importance.csv
    ├── holdout_predictions.csv
    ├── models/
    │   └── RandomForest.pkl
    └── plots/
        ├── predicted_vs_actual.png
        ├── residuals.png
        ├── importance.png
        ├── correlation.png
        └── confusion_matrix.png        (classification only)
```

File	Format	Contents and use
run.json	JSON	Complete record: app version, timestamp, source file path, **data hash**, row count, features (numeric/categorical split), target, task, full config (folds, CV scheme, seed, search budget, holdout, balancing), baseline, every candidate's scores and parameters, chosen and best model, recommendation text, importances, coefficients, plot paths, saved models, and the environment (Python, numpy, pandas, scikit-learn, matplotlib versions).
report.md	Markdown	The same story as readable text: what was asked, the data summary, validation scheme, comparison table, recommendation with justification, importance table, chosen hyperparameters, environment, and a table of the files produced. Attach this to reviews.
metrics.csv	CSV	The comparison table exactly as displayed, including the baseline row.
feature_importance.csv	CSV	Permutation importance per feature with its spread. Negative = harmful.
holdout_predictions.csv	CSV	One row per holdout case: the features, the `actual` value and the `prediction`. The raw material for your own error analysis.
models/*.pkl	joblib pickle	The complete pipeline — imputation, encoding, scaling and estimator — plus metadata (target, features, numeric/categorical split, known categories, training ranges, classes, metrics, config). This is what the Predict and Optimise tabs load.
plots/*.png	PNG	Predicted-vs-actual, residuals, importance bar chart, correlation heatmap; confusion matrix for classification.

> WHY IT MATTERS
>
> The `.pkl` contains the preprocessing. That is the single difference between this program and a script that "just fits a model": there is no way for a future prediction to skip a transformation, and no chance that the median used to fill a blank is computed from the new data instead of the training data.

25.1 Other files the program writes

File	When
`<input>_predictions.csv`	Predict tab, when you leave the output path blank.
`<input>_features.csv`	Molecules tab, after generating descriptors or fingerprints.
`optimised_candidates.csv`	`optimise` command without `-o`.
`<input>_applicability_domain.csv`	`smiles --ad-against` or the AD button.
`MLWorkbench_crash.log`	Only if an unexpected error occurs; written to your user folder, with the traceback.
`selftest.json`	Only with `--selftest`: a machine-readable proof the installation works.

CHAPTER 26

Metric reference

26.1 Regression

Metric	Definition in words	Range	Read it as
R^2	Fraction of the target's variance explained, relative to always predicting the mean.	$-\infty$ … 1.0	0 = no better than the mean; 1 = perfect. Negative = worse than the mean (common for the baseline and for a bad model).
MAE	Mean of |prediction − actual|.	≥ 0, target units	"On average we are off by 2.6 percentage points." The most human-readable error.
RMSE	Square root of the mean squared error.	≥ 0, target units	Same units as MAE but dominated by large misses. RMSE ≫ MAE ⇒ inspect outliers.
MAPE	Mean of |error| / |actual|, as a percentage.	≥ 0 %	Scale-free comparison across targets. Unstable when actual values approach zero.
residual_std	Standard deviation of the errors.	≥ 0	Stored in the model; a useful width for a prediction interval (±2 × this covers roughly 95 %).

26.2 Classification

Metric	Definition in words	Read it as
accuracy	Share of rows classified correctly.	Compare against the majority-class rate, never in isolation.
balanced accuracy	Accuracy computed per class, then averaged.	The fair version when classes are unbalanced.
precision (macro)	For each class, correct predictions ÷ predictions of that class; averaged.	"When it says A, how often is it right?"
recall (macro)	For each class, correct predictions ÷ actual members; averaged.	"Of the real As, how many did it find?"
F1 macro	Harmonic mean of precision and recall, averaged over classes.	**The primary metric here.** Punishes both misses and false alarms, and gives a rare class the same weight as a common one.
F1 weighted	F1 averaged with class frequency as the weight.	Closer to what happens operationally when classes are unbalanced.
ROC AUC (ovr)	Probability that a random positive outranks a random negative, one class versus the rest.	0.5 = no discrimination, 0.9+ = strong. Threshold-free.

26.3 The baseline row

Task	Baseline behaviour	Expected values
Regression	Always predicts the training mean.	$R^2 \approx 0$ in theory, slightly negative in practice (−0.14 in the tutorial run); MAE ≈ the target's standard deviation × 0.8.
Classification	Always predicts the majority class.	Accuracy = majority rate (0.518 for `grade`); macro F1 ≈ 0.2, because the other classes score zero.

CHAPTER 27

Troubleshooting: 24 problems and their fixes

The file will not load

Symptom	Cause and fix
“Could not read file.csv as CSV”	The file is probably not delimited text — an `.xlsx`, a saved web page, or a file with a different extension. Re-export from Excel as “CSV UTF-8”.
Everything lands in one column	The separator is unusual (pipe, multiple spaces). Save it again with commas, or open it in Excel and use Save As → CSV.
Column names have odd characters (ï»¿, ?)	An encoding mismatch. The program strips a UTF-8 BOM automatically; if names still look wrong, re-export as UTF-8 from the source system.
Numbers appear left-aligned and the model is terrible	The column is text (decimal commas, thousands separators, or a stray apostrophe). The health check usually flags it; fix the file so the column is numeric.

The health check complains

Finding	What to do
`target-missing` (error)	Remove the row, fill the value from your records, or move that row into a separate predict file.
`one-class` (error)	Every row has the same label — there is nothing to learn. Check the column you selected, or filter the file.
`duplicate-columns` (error)	Two columns share a name; rename one in the file. Any tool that reads the file will otherwise behave unpredictably.
`id-columns`	Untick the column. Keep it in the file for reference if you like.
`constant`	The column never varies. Untick it; it cannot help.
`rare-class`	Collect more examples of that class, merge it with a neighbour, or accept that the model will not predict it reliably — and say so.
`whitespace-labels` / `case-labels`	Clean the text in the source file (TRIM / UPPER). Fixing it in the file is better than fixing it in a model nobody else knows about.
`redundant-features`	Keep one of the pair. Interpretability improves; tree scores barely move.

Training

Symptom	Cause and fix
“Every model failed”	The log above the error names the reason — usually a target that is entirely empty, a single row, or a text target with one class. Run `audit` first next time.
A model scores far below the others	Normal. Naive Bayes or a single tree on non-linear data will trail a forest. Read the table, do not prune it.
Every model scores ~1.00	A leaked column is almost certainly present: the target itself, a co-measured value, or a per-row identifier. Untick it and re-run.
Training is very slow	SupportVector and MLP dominate. Set the search budget to 0, reduce folds, or use HistGradientBoosting for large data.
Results identical after changing a setting	You changed something the run does not use — for example the group column while the scheme is still `kfold`, or class balancing for a regression task.
Window unresponsive during a run	Expected only for a moment at the plotting stage. Never close the window: the folder is written at the end.

Prediction

Symptom	Cause and fix
“Missing columns: …”	The new file lacks a feature the model needs. Either add the column or retrain without it. Do not proceed with a model that is fed imputed values for a live variable.
“Unseen categories”	A text value (new reactor, new solvent) was not in the training data. Predictions still produce numbers, but for those rows they are guesses. Consider retraining with the new level included.
EXTRAPOLATION warnings	Rows are outside the training range. Report them separately, or experiment in that region before trusting the numbers.
Predictions look shifted compared with an earlier run	You are probably using a different model file. Check the model name in the report panel, and compare `run.json` if in doubt.
Output has no probability columns	The classifier cannot produce probabilities (LinearSupportVector, for example). Use Logistic, RandomForest, GradientBoosting or SupportVector if you need them.

Optimise and Molecules

Symptom	Cause and fix
“Not features of this model: […]”	You are trying to vary a column the model was not trained on. Retrain with it, or remove it from the grid.
“Grid too large”	Above 2 million combinations. Reduce the number of steps or the number of varied features.
Every top row is amber	The optimum sits on the boundary of your search space. Narrow the space around it, or run an experiment there — do not extrapolate.
“rdkit is required” / Molecules tab empty	You are running from source without RDKit installed: `pip install rdkit`. The shipped exe includes it.
SMILES rows come back with empty features	Those strings could not be parsed. The report names the row numbers; check for typos, salts written oddly, or non-SMILES text in the column.

The program itself

Symptom	Cause and fix
Nothing happens on double-click	An unsigned exe silently blocked by SmartScreen or antivirus, or a copy that never finished (a truncated 113 MB file often refuses to start with no message at all). Compare the file size — 118,498,091 bytes — or check the SHA-256 in section 10.4, then try launching from a console to see the error text.
“Unexpected error” dialog	The full traceback is in `MLWorkbench_crash.log` in your user folder. Send that file with a description of the click that caused it.
Anything at all is broken	Run `MLWorkbench.exe --selftest C:\temp\mlw_check`. If it writes `selftest.json` with `"ok": true`, the installation is healthy and the problem is data-specific; if it fails, replace the exe with a fresh copy.

CHAPTER 28

The data-preparation checklist

Ten minutes here saves an hour of confusion and, more importantly, prevents a result you cannot defend.

28.1 Before you open the program

- ☐ One row = one independent observation. If several rows belong to one unit, you have a group column and must say so.
- ☐ The target column exists, is measured consistently, and has no gaps you cannot explain.
- ☐ Every feature would be known at prediction time. This is the leakage test; do it explicitly.
- ☐ Identifiers are named clearly (`id`, `batch`, `run_no`) so you remember to untick them.
- ☐ Units are consistent within a column — no mixing bar and psi, or °C and °F.
- ☐ Headline columns are named without spaces where possible; the program handles them either way, but other tools are less forgiving.
- ☐ The file is saved as CSV UTF-8 with a single header row and no merged cells, no totals row, no blank rows in the middle.

28.2 In the program

- ☐ Health check reviewed line by line; every warning consciously accepted or fixed.
- ☐ Identifier and redundant columns unticked.
- ☐ Target set; the detected task matches your intent.
- ☐ Row count is enough for the number of features (tens of rows per feature is a rough floor).
- ☐ CV scheme matches the data structure: independent / grouped / time-ordered.
- ☐ Holdout ticked, seed recorded.

28.3 After training

- ☐ Baseline read; gain over baseline quoted.
- ☐ Spread read and compared with the differences between models.
- ☐ Holdout score inspected once, after the choice was made.
- ☐ Importance ranking sanity-checked against process knowledge — if temperature scores zero, be suspicious of the data, not pleased with the model.
- ☐ Run folder archived with a one-line note about the question it answered.

CHAPTER 29

Glossary

Baseline

The trivial model: always predict the mean (regression) or the majority class (classification). Every honest comparison includes it.

Bias (in the statistical sense)

Error that does not go away with more data — the model family is too simple for the pattern.

Boosting

Fitting many small models in sequence, each correcting the previous ones' errors. Powerful, slower, easier to overfit than averaging.

Bagging / forest

Fitting many models on random subsets and averaging. RandomForest and ExtraTrees work this way.

Categorical feature

A text or label column (`reactor=R2`). Converted internally to yes/no dummy columns.

Classification

Predicting a category. Judged by accuracy, balanced accuracy, F1.

Class imbalance

One class much rarer than another. Makes plain accuracy misleading; use F1 macro, balanced accuracy, and class weighting.

Confusion matrix

Table of predicted versus actual classes. The off-diagonal cells are the specific mistakes.

Cross-validation (CV)

Splitting the data into folds, training on all but one and scoring on the rest, repeatedly. Gives an average score and its spread.

Data leakage

Information in the features that would not be available at prediction time. Produces excellent scores and useless models.

Data hash

A short fingerprint of the exact CSV used, stored in `run.json` . Detects "same settings, different data".

Descriptor

A number computed from a molecule's structure (weight, logP, ring count).

Extrapolation

Predicting outside the range of the training data. Flagged, never silently trusted.

Feature

An input column. Also: predictor, independent variable, X.

Fingerprint

A bit-vector encoding which substructures a molecule contains.

Fold

One split of a cross-validation.

Group CV

Cross-validation that keeps all rows of one unit (batch, patient) on the same side of the split.

Holdout

Rows locked away before training and used once for a final unbiased score.

Hyperparameter

A setting the model family has but the fitting does not learn, such as tree depth or penalty strength. Tuned by the search budget.

Imputation

Filling missing cells — median for numbers, most frequent for text — using values stored inside the model.

MAE / RMSE / MAPE

Average error, error with large misses punished, and error as a percentage.

Model

Here: the saved object containing preprocessing plus estimator, in a `.pkl` file.

One-hot encoding

Turning a categorical column into several 0/1 columns. Done inside the model.

Overfitting

Learning the noise in the training data. Shows up as a good CV score and a much worse holdout score, or as a large spread.

Permutation importance

How much the score drops when a feature's values are shuffled. The importance shown on the Results tab.

Pipeline

The chained sequence impute → encode → scale → model that is saved as one object.

R^2

Fraction of the target's variance explained, relative to predicting the mean.

Random seed

A number fixing all random choices, making runs reproducible.

Regression

Predicting a number. Judged by R^2, MAE, RMSE.

Residual

Actual minus predicted, per row. Plotted to reveal structure in the errors.

SMILES

A text notation for a molecular structure, e.g. `CCO` for ethanol.

Standardisation

Rescaling features to comparable ranges, done inside the model for the algorithms that need it.

Stratified split

A fold split that preserves class proportions. Automatic for classification.

Target

The column being predicted. Also: label, dependent variable, y.

Test set / holdout

Same thing in this documentation: rows never used for tuning.

TimeSeriesSplit

Expanding-window cross-validation for chronological data.

Train set

The rows a model is allowed to learn from.

Variance
Sensitivity to the particular sample. Shows up as a wide spread between folds.

APPENDIX A

The sample datasets, column by column

A.1 `reactor_yield.csv` — 140 rows × 6 columns

Column	Type	Range	Meaning / role
`temperature_C`	numeric	60.2 … 109.8	Reactor temperature. Feature.
`pressure_bar`	numeric	1.08 … 4.89	Pressure. Feature.
`catalyst_wt`	numeric	0.58 … 2.99	Catalyst charge, wt %. Feature.
`residence_min`	numeric	22 … 120	Residence time, minutes. Feature.
`yield_pct`	numeric	32.87 … 72.46, mean 54.48	**Typical regression target.**
`purity_pct`	numeric	—	Measured after the run — **untick it** for a pre-run model.

A.2 `reactor_predict.csv` — 10 rows × 4 columns

The four features only, no target: the file you feed to the Predict tab.

A.3 `reactor_grade.csv` — 140 rows × 8 columns

Column	Type	Levels / range
`temperature_C`, `pressure_bar`, `catalyst_wt`, `residence_min`	numeric	as above
`reactor`	categorical	R1 (64), R2 (49), R3 (27)
`solvent`	categorical	water (76), ethanol (39), toluene (25)
`shift`	categorical	day (91), night (49) — try excluding it
`grade`	categorical	A (49), B (72), C (19) — **target**

A.4 `molecules.csv` — 34 rows × 3 columns

`name` (text), `SMILES` (structure), `logS` (measured aqueous solubility — a regression target). Run it through the Molecules tab before training, then untick `name` and `canonical_smiles`.

A.5 Guide-only fixtures

File	Purpose
`docs/guide_assets/messy_demo.csv`	12-row file with nine deliberate data problems, for chapter 13.
`docs/guide_assets/daily_output.csv`	360-row, two-line, 180-day production series for chapter 20.

APPENDIX B

Keyboard, menus and file locations

Action	How
Open a CSV	**Ctrl+O**, or File → Open data CSV…, or drag a CSV onto `MLWorkbench.exe`, or Open with from Explorer.
Open the runs folder	Bottom-right button, or File → Open run folder.
Quick guide / About	Help menu.
Close	File → Exit, or the window's close button.
Scroll a list	Mouse wheel over the list, or drag the scrollbar.
Sort a results column	Click its header; click again to reverse.

Location	Contents
`C:\Users\<you>\MLWorkbench_runs`	All run folders (default).
`C:\Users\<you>\MLWorkbench_crash.log`	Written only on an unexpected error.
Next to the input CSV	`*_predictions.csv`, `*_features.csv`, `*_applicability_domain.csv` when you leave the output path blank.
Next to the exe	Nothing. The program writes only where you tell it to.

APPENDIX C

How this program was built and verified

What is inside the executable, how it was tested, and the limits you should know before trusting it with a decision.

C.1 The stack

Component	Version
Python	3.11.9 (embedded in the exe)
scikit-learn	1.5.2
numpy / pandas	1.26.4 / 2.2.3
matplotlib	3.9.2
RDKit	2026.09.1 (Molecules tab)
Packaging	PyInstaller 6.22, single-file build
Optional	xgboost — **not** bundled; install it only in a source installation

C.2 The self-test

The shipped binary responds to `--selftest`, which exercises the whole engine without a window: generate data, train with cross-validation and a holdout, save and reload the model, predict, optimise a grid, and compute molecular descriptors. The build that produced this guide returned:

```
{
  "ok": true,
  "train":    { "chosen": "RandomForest", "rows": 140 },
  "predict":  { "rows": 5, "column": "predicted_yield",
                "values": [30.004, 40.654, 47.116, 43.42, 38.614] },
  "optimise": { "candidates": 50, "best": 53.99 },
  "molecules":{ "rows": 3, "columns": 16 },
  "environment": { "python": "3.11.9", "numpy": "1.26.4", "pandas": "2.2.3",
                   "scikit-learn": "1.5.2", "matplotlib": "3.9.2" }
}
```

If you suspect an installation problem, run it yourself: `MLWorkbench.exe --selftest C:\temp\mlw_check`. A file with `"ok": true` means the program is healthy.

C.3 The eight gaps this program was built to close

1. **Preprocessing lives inside the model**, so training and prediction cannot drift apart.
2. **A written record of every run**: settings, seed, versions and a hash of the exact data.

3. **No “commit” step**: the target and feature selection are the model, not a separate mode you can forget to apply.
4. **The baseline is always displayed**, and the gain over it is stated.
5. **One target per run**, so a result always belongs to exactly one question.
6. **A data audit** for leakage, identifiers, duplicates and rare classes before anything is fitted.
7. **Schema validation and version metadata on prediction**, including extrapolation flags.
8. **Scriptable from the command line**, with encoding and separator detection, and settings that need no administrator rights.

C.4 Known limits — read this before using a result

Limit	Consequence
Supervised learning only	Every run needs a target column. Clustering, anomaly detection and dimensionality reduction are out of scope.
One target at a time	Ten targets means ten runs — which keeps each result attributable.
Tabular data only	No images, spectra, audio or text-as-text. Convert to numeric features first.
Correlation, not causation	Importance ranks what the model uses. Only a designed experiment establishes cause.
Fixed model families	The catalogue in chapter 24 is what you get; no custom estimators from the GUI.
2 million candidate limit	The optimiser refuses grids larger than that rather than freezing your machine.
Pickle compatibility	A `.pkl` is tied to the library versions recorded in it. Keep the run folder, and retrain rather than moving models between very different versions.
No built-in scheduling	Automation is the command line plus Windows Task Scheduler; see chapter 21.

FINAL TIP

The most valuable habit in this whole guide is the least technical one: write down the question before you click, and write down the answer — with its baseline and its spread — before you close the laptop. A model is a sentence about your process, and sentences need to be written down to survive.